

SlabHash — A Source-Code-Driven Guide

A reverse-engineering walkthrough of owensgroup/SlabHash (master branch)

Prepared September 2026

Table of Contents

- 0. How to Use This Document
- 1. The Big Picture
 - 1.1 What SlabHash is
 - 1.2 The problem it solves
 - 1.3 Why a normal CPU-style hash table is not sufficient here
 - 1.4 The key idea: warp-synchronized, slab-based separate chaining
 - 1.5 What “slab” means in this implementation
 - 1.6 Why the implementation is warp-oriented
 - 1.7 How the GPU is involved
 - 1.8 A simple mental model of the whole implementation
- 2. Repository Structure
 - 2.1 Top-level layout
 - 2.2 The role of each major directory
 - 2.3 Inside `src/`: how the files relate to each other
- 3. Architecture
 - 3.1 The real ownership graph (built from the source, not assumed)
 - 3.2 Full ownership diagram
 - 3.3 Which class owns what — summary table
 - 3.4 Host objects vs. GPU-thread-visible objects
 - 3.5 Where memory is allocated, and where entries actually live
 - 3.6 How the allocator interacts with the hash table, at the architecture level
- 4. Core Data Structures
 - 4.1 `key_value_pair<KeyT, ValueT>`
 - 4.2 `concurrent_slab<KeyT, ValueT>` — the `ConcurrentMap` slab layout
 - 4.3 `key_only_slab<KeyT>` and `phase_concurrent_slab<KeyT, ValueT>` — the other two layouts
 - 4.4 `SlabHashTypeT` and the traits classes
 - 4.5 `SlabAllocLightContext` — the allocator’s per-warp state
- 5. The Slab Allocator, Deeply
 - 5.1 Where the allocator is defined
 - 5.2 The three-level memory hierarchy: super-blocks, memory blocks, memory units
 - 5.3 The 32-bit allocator address format
 - 5.4 Turning an address into a real GPU pointer
 - 5.5 How a GPU warp obtains a slab: `warpAllocate`
 - 5.6 Choosing (and re-choosing) a resident memory block
 - 5.7 Why every lane calls `warpAllocate`, even though only one slab comes out
 - 5.8 Freeing a slab: `freeUntouched`
 - 5.9 Bulk allocation: `warpAllocateBulk`
 - 5.10 How `GpuSlabHashContext` and `GpuSlabHash` interact with the allocator — recap
- 6. The Hash-Table Implementation (`GpuSlabHash`, `ConcurrentMap`)
 - 6.1 Initialization
 - 6.2 Buckets: how they are created, and how a key maps to one
 - 6.3 Collisions and how slabs are attached to buckets
 - 6.4 How keys/values are stored
 - 6.5 Insert — the actual code path
 - 6.6 Search — the actual code path
 - 6.7 Delete — the actual code path
 - 6.8 Count — a fourth operation not requested by name but present in the code
 - 6.9 Does the table grow dynamically?
- 7. Warp-Level Execution
 - 7.1 `tid` and `laneId` — computed identically everywhere
 - 7.2 The `work_queue` ballot loop — the core cooperative pattern
 - 7.3 What each lane is doing — concretely
 - 7.4 Synchronization primitives used, and why
 - 7.5 Why a warp, not independent threads — summarized
- 8. `GpuSlabHashContext` in Detail
 - 8.1 Why it exists
 - 8.2 How it differs from `GpuSlabHash`
 - 8.3 Members
 - 8.4 Methods it exposes
 - 8.5 Why GPU kernels use the context (not `GpuSlabHash` directly)
 - 8.6 What memory it owns vs. references
 - 8.7 How it is constructed and passed to kernels
- 9. The SlabHash Variants

- 9.1 `ConcurrentMap` — fully implemented (covered in §6 and §7)
- 9.2 `ConcurrentSet` — implemented, but with real gaps
- 9.3 `PhaseConcurrentMap` — declared as a type tag, but not implemented
- 9.4 Variant comparison summary
- 10. Tracing an Actual CUDA Kernel, Line by Line
 - 10.1 The kernel itself
 - 10.2 Following the call into `GpuSlabHashContext::searchKey`
 - 10.3 The full chain, in one diagram
- 11. Memory Layout
 - 11.1 The two GPU allocations, side by side
 - 11.2 Bucket storage
 - 11.3 Slab storage (chained/overflow slabs)
 - 11.4 Key/value storage within one slab
 - 11.5 Pointers / indices
 - 11.6 Allocator metadata
 - 11.7 GPU memory ownership recap
- 12. Concurrency
 - 12.1 Linking a new slab onto a chain: `atomicCAS` on the next-pointer word
 - 12.2 Inserting a key/value pair: 64-bit `atomicCAS`
 - 12.3 Deleting a key: 64-bit `atomicExch`
 - 12.4 Allocating a slab: `atomicCAS` on a bitmap word
 - 12.5 Warp-collective ballots/shuffles themselves are a synchronization mechanism
 - 12.6 Memory ordering
- 13. The Benchmark Code
 - 13.1 How benchmarks are configured
 - 13.2 How the number of buckets is chosen: `expected_chain`
 - 13.3 What operations are measured, and how
 - 13.4 What the different benchmark modes mean
 - 13.5 Which functions actually execute the hash-table operations
 - 13.6 `bencher.py`
- 14. The Tests
 - 14.1 `test/cmap_test.cu` — the gtest suite (the primary correctness tests)
 - 14.2 The standalone (non-gtest) test executables
 - 14.3 What running the tests actually exercises, end to end
- 15. The Build System
 - 15.1 The root `CMakeLists.txt`
 - 15.2 `src/CMakeLists.txt` — defined, but not actually built by the root project
 - 15.3 `SlabAlloc/CMakeLists.txt` — likewise not invoked from the SlabHash root
 - 15.4 `test/CMakeLists.txt` and `bench/CMakeLists.txt`
 - 15.5 How the repository is actually built and run
- 16. Complete Call/Data-Flow Walkthroughs
 - 16.1 Insertion: `Application` → `GpuSlabHash API` → `Context` → `Kernel` → `Warp` → `Bucket` → `Slab` → `Key/Value`
 - 16.2 Deletion
 - 16.3 Search, restated compactly for comparison (full trace in §10)
 - 16.4 One structural observation across all three operations
- 17. Recommended Reading Order
- 18. A Note on the Code Snippets in This Document
- 19. SlabHash — Complete Mental Model

0. How to Use This Document

This document is a guided, source-code-driven walkthrough of the **SlabHash** repository (<https://github.com/owensgroup/SlabHash>, `master` branch, plus its required **SlabAlloc** submodule from <https://github.com/owensgroup/SlabAlloc>). Every claim below is grounded in a specific file and function in the repository. Code snippets are copied verbatim from the source tree (paths given at the top of each snippet); nothing has been rewritten or invented. Where the source code itself is ambiguous or leaves something unspecified, that is called out explicitly rather than guessed at.

The repository is small (roughly 30 CUDA/C++ header and source files, ~2,500 lines total once the **SlabAlloc** submodule is included), but it is dense: almost every line matters, and the concurrency logic is subtle. This guide follows the structure that is most useful for actually opening the repository and reading it afterward: big picture first, then repository layout, then architecture, then a deep dive into data structures, the allocator, the hash table operations, warp mechanics, variants, kernels, memory layout, concurrency, benchmarks, tests, and the build system — ending with a single end-to-end mental model and a recommended reading order.

1. The Big Picture

1.1 What SlabHash is

SlabHash is a **dynamic hash table that lives entirely on the GPU and is built, queried, and modified by GPU threads themselves** — not by the CPU issuing one operation at a time. The repository’s own README states this plainly:

“A warp-oriented dynamic hash table for GPUs”

and traces back to the original research: *A Dynamic Hash Table for the GPU* (Ashkiani, Farach-Colton, Owens — IPDPS’18). The code in this repository is a refactored version of that paper’s implementation ([README.md](#), lines 1–8).

Concretely, SlabHash lets you push millions of key-value pairs (or just keys, for the set variant) onto the GPU and then run **bulk builds, bulk/individual searches, deletions, and counts as CUDA kernels**, with all the bucket traversal, collision resolution, and dynamic memory allocation for growing chains happening inside `_device_` code — no round-trips to the CPU per operation.

1.2 The problem it solves

A hash table needs two things that are easy on a CPU but hard on a GPU:

1. **Concurrent, dynamic memory allocation for collision chains.** When two keys collide, something has to grow — a new bucket, node, or bucket segment has to be allocated. On a CPU this is `malloc/new`. On a GPU, thousands of threads across many warps might need to allocate memory in the same microsecond, and a naive global allocator becomes a serialization bottleneck.
2. **Coordinated, race-free updates from thousands of threads at once.** CPU hash tables typically use one lock (or a handful of locks) per bucket. A GPU warp has 32 threads executing in lockstep; if a normal CPU-style “each thread does its own probing and locking” design were ported directly, the threads inside a single warp would immediately diverge and serialize against each other, defeating the SIMT execution model.

SlabHash’s answer to both problems is visible directly in the source: a purpose-built **slab-based bump-pointer/bitmap allocator** (`SlabAlloc`, see §5) for problem (1), and a **warp-cooperative “ballot loop” protocol** baked into every hash-table operation (see §7) for problem (2).

1.3 Why a normal CPU-style hash table is not sufficient here

A textbook CPU hash table (e.g., `std::unordered_map`, or a simple separate-chaining table with `malloc`) makes assumptions that don’t hold on a GPU:

- It assumes each **thread of control is independent** and can take its own branch, follow its own pointer chain, and block on a lock without affecting other threads’ throughput. On a GPU, the 32 threads of a warp execute in lockstep (SIMT); if thread 3 is chasing a 5-node chain while thread 17 found its key immediately, thread 17 either idles or the warp has to serialize the two paths — throughput collapses if this happens per-key instead of per-warp.
- It assumes **`malloc/new` is a cheap, general-purpose operation**. GPU global `malloc` inside kernels is notoriously slow and contention-prone at thousands-of-threads scale, precisely because it’s built for a general-purpose, largely-serial allocation pattern.
- It assumes **memory layout doesn’t need to match the hardware’s access granularity**. GPUs fetch memory in coalesced transactions; a hash table whose nodes are scattered structs-with-pointers, one CPU-cache-line at a time, throws away most of the GPU’s memory bandwidth.

SlabHash’s design directly targets all three assumptions: it replaces per-thread independence with per-warp cooperation (§7), replaces general `malloc` with a **warp-synchronized bitmap allocator that hands out one 128-byte unit per warp-vote** (§5), and replaces scattered nodes with **32-word “slabs” that are read by an entire warp in a single coalesced 128-byte transaction** (§4, §11).

1.4 The key idea: warp-synchronized, slab-based separate chaining

SlabHash is a **separate-chaining hash table**: each bucket is the head of a singly linked list of fixed-size nodes (“slabs”). What’s unusual is *how* that chain is read and extended:

- Every slab, whether it is the bucket’s base slab or a chained slab allocated later, is **exactly 128 bytes = 32 × 4-byte words** — the same size as one memory transaction serviced by one full warp of 32 threads, each reading/writing one word. This is enforced directly in the code with a compile-time assert:

```
// src/concurrent_map/cmap_class.cuh - GpuSlabHash constructor
assert(sizeof(typename ConcurrentMapT<KeyT, ValueT>::SlabTypeT) ==
       (WARP_WIDTH * sizeof(uint32_t)) &&
       "A single slab on a ConcurrentMap should be 128 bytes");
```

- All 32 threads of a warp **cooperate on one slab at a time**, using `__ballot_sync` to agree on which lane’s key is currently “active,” fetching the entire 128-byte slab with one coalesced load (lane *i* reads word *i*), and using warp shuffles (`__shfl_sync`) to broadcast the values every lane needs to compare against, without ever touching shared or global scratch memory for coordination.
- When a chain needs to grow, one lane of the warp calls into the allocator (`allocateSlab`, §5) to get a fresh 128-byte unit and links it in with a single `atomicCAS` on the “next pointer” word.

This is exactly the “warp-cooperative work sharing” (WCWS) strategy the code comments name directly, e.g. in `src/concurrent_map/warp/insert.cuh`:

```
/*
 * each thread inserts a key-value pair into the hash table
 * it is assumed all threads within a warp are present and collaborating with
 * each other with a warp-cooperative work sharing (WCWS) strategy.
 */
```

1.5 What “slab” means in this implementation

A “slab” is a **fixed-size, 128-byte block of GPU memory laid out as 32 consecutive `uint32_t` words**, used as one node of a bucket’s collision chain. There is more than one *layout* of slab (see §4 and §9), but they are all still 32 words / 128 bytes. For the main `ConcurrentMap` variant, a slab is defined by the `concurrent_slab` struct:

```
// src/slab_hash_global.cuh
template <typename KeyT, typename ValueT>
struct __align__(32) concurrent_slab {
    static constexpr uint32_t NUM_ELEMENTS_PER_SLAB = 15u;
    key_value_pair<KeyT, ValueT> data[NUM_ELEMENTS_PER_SLAB];
    uint32_t ptr_index[2];
};
```

15 key/value pairs (2 words each = 30 words) plus a 2-word `ptr_index` tail = 32 words = 128 bytes. Only the **last** of those two tail words (lane 31) is actually used by the current code as the “pointer to the next slab” — see §4.4 and §11 for the exact bit-for-bit layout, including a slot the code declares but never reads or writes.

“Slab” therefore means something more specific than “a chunk of memory”: it is *the unit of warp-cooperative access* — one slab is read or written as a single unit by an entire warp, one lane per word.

1.6 Why the implementation is warp-oriented

Every public entry point into the hash table (`insertPair`, `searchKey`, `deleteKey`, `countKey`, ...) is documented and implemented as an operation that **32 threads perform together**, not 32 independent operations. This is because:

1. A slab is exactly 32 words — a perfect match for a warp’s 32 lanes, so a whole slab is read with one coalesced transaction if all 32 lanes participate.
2. `__ballot_sync`, `__shfl_sync`, and `atomicCAS` are warp/thread primitives the code uses to let all 32 lanes agree on *whose* key is currently being processed (see the `work_queue` pattern in §7), so that only one lane’s key is “in flight” against a given slab at a time, but all 32 lanes physically fetch and inspect that slab together.
3. Because the allocator (§5) is also warp-synchronized (`warpAllocate` requires all 32 lanes to call it together and cooperatively pick a free slot via ballot), a slab can only sensibly be allocated by a full warp, reinforcing the warp as the natural unit of work throughout the code.

1.7 How the GPU is involved

Everything after the CPU-side `GpuSlabHash` constructor and the bulk `cudaMemcpy` calls that move keys/values to the device runs as CUDA kernels:

- **Building** the table: `build_table_kernel` / `build_table_with_unique_keys_kernel` (`src/concurrent_map/device/build.cuh`).
- **Searching**: `search_table` / `search_table_bulk` (`src/concurrent_map/device/search_kernel.cuh`).

- **Deleting:** `delete_table_keys` (`src/concurrent_map/device/delete_kernel.cuh`).
- **Counting:** `count_key` (`src/concurrent_map/device/count_kernel.cuh`).
- **Mixed batches:** `batched_operations` (`src/concurrent_map/device/concurrent_kernel.cuh`).

Each kernel launches one thread per key/query (grouped into warps of 32), computes a `laneId`, computes which bucket a key hashes to, and then calls into `GpuSlabHashContext` member functions (`insertPair`, `searchKey`, ...) that do the actual warp-cooperative traversal of the slab chain — entirely inside `__device__` code, entirely in GPU global memory. The CPU's job, visible in `src/concurrent_map/cmap_implementation.cuh`, is reduced to computing a grid size and launching the kernel:

```
// src/concurrent_map/cmap_implementation.cuh
template <typename KeyT, typename ValueT>
void GpuSlabHash<KeyT, ValueT, SlabHashTypeT::ConcurrentMap>::searchIndividual(
    KeyT* d_query,
    ValueT* d_result,
    uint32_t num_queries) {
    CHECK_CUDA_ERROR(cudaSetDevice(device_idx_));
    const uint32_t num_blocks = (num_queries + BLOCKSIZE_ - 1) / BLOCKSIZE_;
    search_table<KeyT, ValueT>
        <<<num_blocks, BLOCKSIZE_>>>(d_query, d_result, num_queries, gpu_context_);
}
```

1.8 A simple mental model of the whole implementation

Before diving into files, here is the mental model this document will justify, piece by piece, with source code:

Host (CPU) side	Device (GPU) side
-----	-----
<code>gpu_hash_table</code>	(nothing runs here directly;
owns arrays of keys/values/queries	it is reached only via kernels)
owns <code>GpuSlabHash*</code>	
owns <code>DynamicAllocatorT*</code> (<code>SlabAllocLight</code>)	
constructs, holds device pointers	
v	
<code>GpuSlabHash<KeyT, ValueT, Type></code>	<code>GpuSlabHashContext<KeyT, ValueT, Type></code>
OWNS <code>d_table_</code> (bucket array, <code>cudaMalloc</code>)	does NOT own memory
OWNS <code>gpu_context_</code> (a <code>GpuSlabHashContext</code>)	holds a raw pointer to <code>d_table_</code>
launches kernels, passing <code>gpu_context_</code>	holds a COPY of the allocator's
by value	context (<code>AllocatorContextT</code>)
	exposes <code>insertPair/searchKey/...</code>
<<<kernel>>>(..., <code>gpu_context_</code>)	these run <code>__device__</code> , per warp
v	
CUDA kernel (e.g. <code>search_table</code>)	
computes <code>tid</code> , <code>laneId</code> , bucket	
calls <code>slab_hash.searchKey(...)</code>	
v	
<code>GpuSlabHashContext::searchKey</code> (warp-cooperative)	
ballot to find whose key is "active" this round	
read one 128-byte slab (bucket's base slab, or a chained slab)	
compare keys via ballot, shuffle out the value if found	
if not found and there's a next-pointer, follow the chain	
v	
Slab chain in GPU global memory (owned by <code>GpuSlabHash</code> 's <code>d_table_</code>	
for the base slabs, and by <code>SlabAllocLight</code> 's super-blocks for all	
chained/overflow slabs)	

The rest of this document expands every arrow above with the exact source that produces it.

2. Repository Structure

2.1 Top-level layout

SlabHash/	
├─ CMakeLists.txt	top-level build configuration
├─ LICENSE	Apache 2.0
├─ README.md	overview, publication link, usage example, benchmark tables
├─ SlabAlloc/	<-- git submodule (owensgroup/SlabAlloc): the allocator
│ └─ CMakeLists.txt	
│ └─ src/	
│ └─ slab_alloc.cuh	the allocator itself (SlabAllocLight[Context])
│ └─ slab_alloc.cu	one-line .cu file that just #includes the .cuh
│ └─ slab_alloc_global.cuh	SlabAllocAddressT typedef + CHECK_ERROR macro
│ └─ test/	
│ └─ test_slab_alloc.cu	a standalone smoke test for the allocator alone
├─ ThirdParty/	
│ └─ googletest/	<-- submodule: gtest, used by test/cmap_test.cu
│ └─ rapidjson/	<-- submodule: used by bench/ to emit JSON results
├─ src/	the hash table itself (this is "SlabHash proper")
│ └─ CMakeLists.txt	
│ └─ CommandLine.h	
│ └─ slab_hash_global.cuh	slab layouts, type tags, global constants
│ └─ slab_hash_helper_methods.cuh	small per-warp key/empty search helpers
│ └─ slab_hash.cuh	the single "master include" for the library
│ └─ slab_iterator.cuh	host/device iterator over a ConcurrentSet
│ └─ gpu_hash_table.cuh	example application-level wrapper class
│ └─ concurrent_map/	the ConcurrentMap variant (most complete)
│ └─ cmap_class.cuh	GpuSlabHash & GpuSlabHashContext declarations
│ └─ cmap_implementation.cuh	host-side member function bodies (kernel launches)
│ └─ device/	__global__ kernels
│ └─ build.cuh	
│ └─ search_kernel.cuh	
│ └─ delete_kernel.cuh	
│ └─ count_kernel.cuh	
│ └─ concurrent_kernel.cuh	
│ └─ misc_kernels.cuh	
│ └─ warp/	__device__ warp-cooperative operation bodies
│ └─ insert.cuh	
│ └─ search.cuh	
│ └─ delete.cuh	
│ └─ count.cuh	
│ └─ concurrent_set/	the ConcurrentSet variant (keys only, no values)
│ └─ cset_class.cuh	
│ └─ cset_implementation.cuh	
│ └─ cset_helper_kernels.cuh	
│ └─ cset_warp_operations.cuh	
├─ bench/	standalone benchmarking executable + Python driver
│ └─ CMakeLists.txt	
│ └─ main_benchmarks.cu	CLI entry point, argument parsing, mode dispatch
│ └─ experiments.cuh	the actual experiment bodies (build/search/etc. + JSON output)
│ └─ batched_data_gen.h/.cpp	random key generation and batch construction
│ └─ bench.py	thin Python wrapper that shells out to the built binary
├─ test/	correctness tests (gtest-based and standalone)
│ └─ CMakeLists.txt	
│ └─ cmap_test.cu	gtest suite: construction, build, search, count, delete
│ └─ test_slab_hash.cu	minimal standalone smoke test (no gtest)
│ └─ concurrent_map.cu	standalone ConcurrentMap exercise (no gtest)
│ └─ concurrent_set.cu	standalone ConcurrentSet exercise (no gtest)
│ └─ iterator_test.cu	exercises SlabIterator by printing the whole table

2.2 The role of each major directory

SlabAlloc/ is *not* part of the SlabHash repository's own git history — `.gitmodules` declares it as a submodule pointing at <https://github.com/owensgroup/SlabAlloc>:

```
[submodule "SlabAlloc"]
  path = SlabAlloc
  url = https://github.com/owensgroup/SlabAlloc
```

This is why the README's compilation instructions start with `git submodule init / git submodule update` (README.md, "Compilation" section) — without that step, `SlabAlloc/src/slab_alloc.cuh` simply does not exist, and nothing in `src/` can compile, because `src/slab_hash_global.cuh` does `#include "slab_alloc.cuh"` unconditionally. **The allocator is a separate, independently versioned project** that SlabHash depends on but does not implement itself. This document treats `SlabAlloc` as first-class material (§5) because the user's question about "the slab allocator" is really a question about this submodule.

`src/` is SlabHash proper: the hash-table data structure, its two fully implemented variants (`concurrent_map`, `concurrent_set`), and the small number of global files that both variants depend on. This is the directory someone would actually vendor into their own CUDA project.

`bench/` is a self-contained benchmarking harness. It is a *consumer* of the `src/` API (via `gpu_hash_table`, see `src/gpu_hash_table.cuh`) — it does not modify or extend the hash table. Its role is to generate large batches of random keys, drive `build/search/delete` calls, time them with CUDA events, and dump results as JSON via `rapidjson`. `bench/bencher.py` is a thin Python convenience layer that just invokes the compiled `build/bin/benchmark` executable with different `-mode` flags and parses/plots the resulting JSON.

`test/` contains both a `gtest`-based correctness suite (`cmap_test.cu`, linked against `ThirdParty/gtest`) and several standalone `main()`-based smoke-test executables that are *not* part of the `gtest` suite (`test_slab_hash.cu`, `concurrent_map.cu`, `concurrent_set.cu`, `iterator_test.cu`). Each becomes its own CMake executable target (§15).

`ThirdParty/` holds two more submodules: `googletest` (only used by `test/cmap_test.cu`) and `rapidjson` (only used by `bench/experiments.cuh` to serialize benchmark results).

`CMakeLists.txt` at the root wires all of this together: it sets CUDA architecture flags, adds `SlabAlloc/src`, `src`, and the two `ThirdParty` include directories to the include path, and adds `test` and `bench` as subdirectories (it does **not** `add_subdirectory(SlabAlloc)` — SlabHash only needs `SlabAlloc`'s headers, which are header-only style aside from the trivial `slab_alloc.cu`; the allocator is compiled directly into whatever target includes `slab_hash.cuh`). Full detail in §15.

2.3 Inside `src/`: how the files relate to each other

The files in `src/` are not independent; they form a strict dependency chain, and the single entry point a user is expected to `#include` is `src/slab_hash.cuh`, whose entire body is a curated list of includes:

```
// src/slab_hash.cuh
#include "slab_hash_global.cuh"           // 1. types, constants, slab layouts
#include "slab_hash_helper_methods.cuh"   // 2. small per-warp search helpers

#include "concurrent_map/cmap_class.cuh"   // 3. class *declarations* (both variants)
#include "concurrent_set/cset_class.cuh"
#include "slab_iterator.cuh"

#include "concurrent_map/warp/delete.cuh"  // 4. __device__ member function *bodies*
#include "concurrent_map/warp/insert.cuh"
#include "concurrent_map/warp/search.cuh"
#include "concurrent_map/warp/count.cuh"
#include "concurrent_set/cset_warp_operations.cuh"

#include "concurrent_map/device/build.cuh" // 5. __global__ kernels
#include "concurrent_map/device/concurrent_kernel.cuh"
#include "concurrent_map/device/delete_kernel.cuh"
#include "concurrent_map/device/misc_kernels.cuh"
#include "concurrent_map/device/search_kernel.cuh"
#include "concurrent_map/device/count_kernel.cuh"
#include "concurrent_set/cset_helper_kernels.cuh"

#include "concurrent_map/cmap_implementation.cuh" // 6. host-side method bodies
#include "concurrent_set/cset_implementation.cuh"
```

This ordering is not arbitrary — it mirrors a real C++ constraint. Because `GpuSlabHashContext<...>::insertPair` etc. are declared (not defined) inside the class body in `cmap_class.cuh`, and their *definitions* live in the `warp/*.cuh` files, the class declaration must be visible first. Likewise, the `__global__` kernels in `device/*.cuh` call `slab_hash.insertPair(...)` etc., so they must come after the warp-level definitions. Finally, `cmap_implementation.cuh` (host-side `GpuSlabHash::buildBulk`, etc.) launches those `__global__` kernels, so it is included last.

`gpu_hash_table.cuh` sits one layer above all of this: it `#include "slab_hash.cuh"` and provides a convenience class (`gpu_hash_table<KeyT, ValueT, SlabHashT>`) that owns both a `GpuSlabHash` and a `DynamicAllocatorT`, and exposes plain-looking methods like `hash_build`, `hash_search`, `hash_delete`. This is the class every test and benchmark in the repository actually instantiates — see §3 for exactly why this ownership split exists.

3. Architecture

3.1 The real ownership graph (built from the source, not assumed)

The prompt's suggested diagram (`Application` → `GpuSlabHash` → `Hash-table implementation` → `Slab allocator` → `GPU memory`) is close, but the actual code has a more specific shape: there are **two parallel object hierarchies** — one for the hash table's *bucket storage*, and one for the *dynamic allocator's overflow storage* — and they are deliberately **not owned by the same object**, even though both end up used together by every GPU thread.

From `src/gpu_hash_table.cuh`:

```
template <typename KeyT, typename ValueT, SlabHashTypeT SlabHashT>
class gpu_hash_table {
...
public:
    // Slab hash invariant
    GpuSlabHash<KeyT, ValueT, SlabHashT>* slab_hash_;

    // the dynamic allocator that is being used for slab hash
    DynamicAllocatorT* dynamic_allocator_;
...
    gpu_hash_table(...)
        : ...
          , dynamic_allocator_(nullptr)
          ... {
        ...
        // allocate and initialize the allocator:
        dynamic_allocator_ = new DynamicAllocatorT();

        // slab hash:
        slab_hash_ = new GpuSlabHash<KeyT, ValueT, SlabHashT>{
            num_buckets_, dynamic_allocator_, device_idx_, seed_, identity_hash_};
        ...
    }
```

So `gpu_hash_table` (the example application layer) is what **owns the allocator** (`new DynamicAllocatorT()`) and then hands a raw pointer to it into the `GpuSlabHash` constructor. `GpuSlabHash` itself does **not** own the allocator — its constructor just stores the pointer:

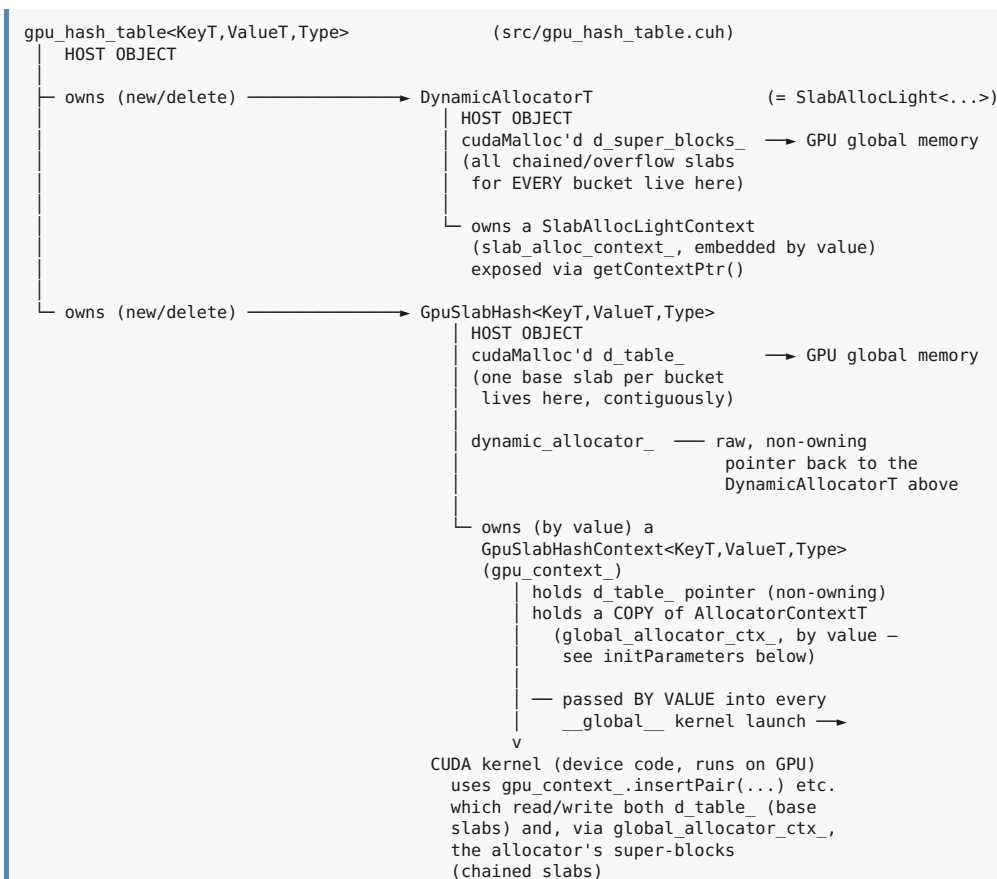
```
// src/concurrent_map/cmap_class.cuh
GpuSlabHash(const uint32_t num_buckets,
            DynamicAllocatorT* dynamic_allocator,
            uint32_t device_idx,
            const time_t seed = 0,
            const bool identity_hash = false)
    : num_buckets_(num_buckets)
    , d_table_(nullptr)
    , slab_unit_size_(0)
    , dynamic_allocator_(dynamic_allocator) // <-- just stores the pointer
    , device_idx_(device_idx) {
...
    // allocating initial buckets:
    CHECK_CUDA_ERROR(cudaMalloc((void*)&d_table_, slab_unit_size_ * num_buckets_));
    ...
}
```

What `GpuSlabHash` *does* own is `d_table_` — the array of **base slabs**, one per bucket, allocated with its own `cudaMalloc` call. This is a completely separate GPU allocation from the allocator's memory pool. The README states this design choice directly:

"Any other similar application level API should also own the dynamic memory allocator that is used by all instances of GpuSlabHash class (here just one)."

In other words: **the allocator is meant to be shared across possibly multiple `GpuSlabHash` instances**, so it is deliberately owned one level up, by whatever application class the user writes (`gpu_hash_table` here is just the example the repo ships).

3.2 Full ownership diagram



The critical wiring step is `GpuSlabHash`'s constructor calling `gpu context .initParameters(...)`:

```
// src/concurrent_map/cmap_class.cuh, end of GpuSlabHash constructor
gpu_context_.initParameters(
    num buckets , hf .x, hf .y, d table , dynamic allocator ->getContextPtr());
```

```
and GpuSlabHashContext::initParameters:
```

```
// src/concurrent_map/cmap_class.cuh
__host__ void initParameters(const uint32_t num_buckets,
                             const uint32_t hash_x,
                             const uint32_t hash_y,
                             int8_t* d_table,
                             AllocatorContextT* allocator_ctx) {

    num_buckets_ = num_buckets;
    hash_x_ = hash_x;
    hash_y_ = hash_y;
    d_table_ =
        reinterpret_cast<typename ConcurrentMapT<KeyT, ValueT>::SlabTypeT*>(d_table);
    global_allocator_ctx_ = *allocator_ctx;    // <-- a COPY, not a pointer
}
```

So `GpuSlabHashContext` does not point at the allocator's context — it **copies** it by value (`AllocatorContextT global_allocator_ctx;` is a plain member, not a pointer). This matters because `GpuSlabHashContext` itself is then passed by value into every kernel launch (see §3.4), so the allocator's context (which itself only contains a raw `uint32_t*` to the GPU memory pool plus a couple of small scalars — see §5.2) is cheaply copyable and safe to duplicate per kernel launch/per thread.

3.3 Which class owns what — summary table

Class	Lives on	Owns	Does not own (references only)
<code>gpu_hash_table</code>	Host	<code>DynamicAllocatorT*</code> , <code>GpuSlabHash*</code> , key/value/query/result device arrays	—
<code>SlabAllocLight</code>	Host object, GPU-resident memory	<code>d_super_blocks_</code> (<code>cudaMalloc</code> 'd GPU memory for all overflow slabs across all buckets)	—
<code>GpuSlabHash<...></code>	Host object, GPU-resident memory	<code>d_table_</code> (<code>cudaMalloc</code> 'd GPU memory for the base slab of every bucket)	<code>dynamic_allocator_</code> (raw pointer, borrowed)
<code>GpuSlabHashContext<...></code>	Host object <i>and</i> copied into device code	Nothing (no <code>cudaMalloc</code> / <code>cudaFree</code> calls anywhere in this class)	<code>d_table_</code> (raw pointer copy), <code>global_allocator_ctx_</code> (value copy of the allocator's context)
<code>SlabAllocLightContext</code>	Host object <i>and</i> copied into device code	Nothing (no allocation calls)	<code>d_super_blocks_</code> (raw pointer copy)

The pattern is consistent and deliberate: **the two classes with `cudaMalloc` calls (`SlabAllocLight`, `GpuSlabHash`) are the only owners; their matching “Context” classes are cheap, copyable, pointer-holding views meant to be passed by value into `__global__` kernels.** This is exactly the distinction the README calls out:

“This class partially owns all the memory allocated on the GPU to actually store all the contents... There is another class, named `GpuSlabHashContext`, which does not own any memory but has all the related member functions to use the data structure itself. The context class is the one which is used by GPU threads on the device.”

3.4 Host objects vs. GPU-thread-visible objects

Only `GpuSlabHashContext` (and, nested inside it, `AllocatorContextT = SlabAllocLightContext`) is ever actually present in `__device__` code. This is visible directly from every kernel signature, e.g.:

```
// src/concurrent_map/device/search_kernel.cuh
template <typename KeyT, typename ValueT>
__global__ void search_table(
    KeyT* d_queries,
    ValueT* d_results,
    uint32_t num_queries,
    GpuSlabHashContext<KeyT, ValueT, SlabHashTypeT::ConcurrentMap> slab_hash) {
```

`slab_hash` here is passed **by value** — CUDA copies this small struct (a few `uint32_t`s, one pointer, and the allocator context) into each thread block's parameter space. Neither `GpuSlabHash` nor `gpu_hash_table` nor `SlabAllocLight` ever appears in a kernel signature anywhere in the repository — a `grep` for these class names inside any `__global__` function confirms this. They are purely host-side orchestration objects: they call `cudaMalloc`, construct a `GpuSlabHashContext`, and launch kernels that only see that lightweight context.

3.5 Where memory is allocated, and where entries actually live

There are exactly two `cudaMalloc` call sites that matter for storing table contents:

1. **`SlabAllocLight`'s constructor** (`SlabAlloc/src/slab_alloc.cuh`): allocates one big flat array, `d_super_blocks_`, sized `SUPER_BLOCK_SIZE * num_super_blocks * sizeof(uint32_t)`. This is where **every slab beyond a bucket's first (base) slab** is allocated from, across *all* buckets and (if the application creates more than one) potentially across multiple hash tables sharing the same allocator.
2. **`GpuSlabHash`'s constructor** (`src/concurrent_map/cmap_class.cuh`): allocates `d_table_`, sized `slab_unit_size * num_buckets` — exactly one 128-byte base slab per bucket, contiguously. This is the **“bucket array”** — bucket *i*'s base slab starts at byte offset `i * 128` inside `d_table_`.

So a bucket's chain is:

```

d_table_[bucket_id] (owned by GpuSlabHash, fixed at construction time)
    |
    | base slab, 128 bytes, always exists even if the bucket is empty
    | (bucket "empty" == all 32 words are 0xFFFFFFFF, from the
    |   cudaMemset(d_table_, 0xFF, ...) in the GpuSlabHash constructor)
    | last word (lane 31) = pointer to the next slab, or
    | EMPTY_INDEX_POINTER (0xFFFFFFFF) if the chain ends here
    |
    v
[overflow slab #1] (owned by SlabAllocLight, allocated on demand by
    | a warp calling allocateSlab() during insertPair)
    | lane 31 = pointer to the next slab, or EMPTY_INDEX_POINTER
    |
    v
[overflow slab #2] ... and so on, however long the chain needs to be
    |
    v
EMPTY_INDEX_POINTER (0xFFFFFFFF) terminates the chain

```

3.6 How the allocator interacts with the hash table, at the architecture level

`GpuSlabHashContext` never calls `cudaMalloc` — instead it holds a **value copy** of `AllocatorContextT` (`global_allocator_ctx_`) and delegates all actual slab acquisition/release to it, through two tiny private wrapper methods:

```

// src/concurrent_map/cmap_class.cuh (private section of GpuSlabHashContext)
__device__ __forceinline__ SlabAllocAddressT
allocateSlab(AllocatorContextT& local_allocator_ctx, const uint32_t& laneId) {
    return local_allocator_ctx.warpAllocate(laneId);
}

__device__ __forceinline__ void freeSlab(const SlabAllocAddressT slab_ptr) {
    global_allocator_ctx_.freeUntouched(slab_ptr);
}

```

Everything about how `warpAllocate` and `freeUntouched` work internally (bitmaps, super-blocks, atomics) is covered in §5. Architecturally, the important point is that `GpuSlabHashContext` treats the allocator purely as a service it calls — it has no idea how memory blocks are organized internally, it just gets back an opaque 32-bit `SlabAllocAddressT` “address” that it can later turn into a real GPU pointer via `getPointerFromSlab`, which itself just forwards to the allocator context:

```

// src/concurrent_map/cmap_class.cuh
__device__ __forceinline__ uint32_t* getPointerFromSlab(
    const SlabAddressT& slab_address,
    const uint32_t laneId) {
    return global_allocator_ctx_.getPointerFromSlab(slab_address, laneId);
}

```

4. Core Data Structures

All of the structures in this section are defined in `src/slab_hash_global.cuh` except where noted. This is deliberate: the repository puts every POD (“plain old data”) layout used by *any* variant into one file, so that `concurrent_map` and `concurrent_set` share a single source of truth for what a “slab” looks like in memory.

4.1 `key_value_pair<KeyT, ValueT>`

```
// src/slab_hash_global.cuh
// only works with up to 32-bit key/values
template <typename KeyT, typename ValueT>
struct key_value_pair {
    KeyT key;
    ValueT value;
};
```

- **Represents:** one key/value entry, stored inline (not as a pointer to a heap object).
- **Members:** `key` and `value`, each expected to be (at most) 32 bits — the comment is explicit about this restriction, and the rest of the code reinforces it everywhere by reinterpreting keys/values as `uint32_t` for atomics and shuffles (see §7, §12).
- **Who creates it:** never constructed on its own — it only ever appears as the element type of the `data[]` array inside `concurrent_slab` (§4.4). No code anywhere calls `new key_value_pair(...)` or similar.
- **Who owns its memory:** whatever owns the `concurrent_slab` it’s embedded in (either `GpuSlabHash::d_table` for a bucket’s base slab, or `SlabAllocLight::d_super_blocks` for a chained slab).
- **Where it exists:** GPU global memory only, as raw bytes — the struct itself is only used device-side (`insertPair`, `searchKey`, etc.). No host code ever reads or writes a `key_value_pair` directly; the host only ever moves flat `KeyT*/ValueT*` arrays across `cudaMemcpy`.

4.2 `concurrent_slab<KeyT, ValueT>` — the `ConcurrentMap` slab layout

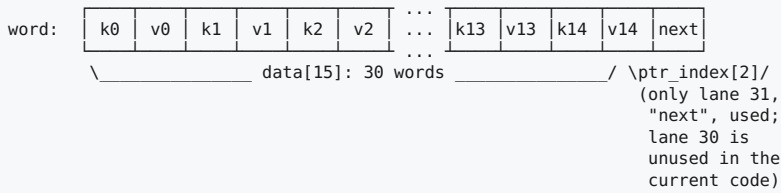
```
// src/slab_hash_global.cuh
template <typename KeyT, typename ValueT>
struct __align__(32) concurrent_slab {
    static constexpr uint32_t NUM_ELEMENTS_PER_SLAB = 15u;
    key_value_pair<KeyT, ValueT> data[NUM_ELEMENTS_PER_SLAB];
    uint32_t ptr_index[2];
};
```

- **Represents:** one node of a bucket’s collision chain in the `ConcurrentMap` variant — either the bucket’s own base slab, or a dynamically allocated overflow slab.
- **Members:**
 - `data[15]`: fifteen `key_value_pair`s, i.e. 15 keys and 15 values, interleaved `key0, value0, key1, value1, ..., key14, value14` — 30 `uint32_t` words total.
 - `ptr_index[2]`: two trailing words. As established in §1.5 and verified by a full-repository search, **only `ptr_index[1]` (the very last word, warp lane 31) is ever read or written** by any insert/search/delete/count code — always through the literal lane index 31 (e.g. `getPointerFromSlab(next, 31)`, `__shfl_sync(0xFFFFFFFF, src_unit_data, 31, 32)`). `ptr_index[0]` (lane 30) is declared but this document could not find any read or write of it anywhere in `src/`. The source gives no comment explaining its purpose, so rather than guess, this is flagged here as an apparently-unused/reserved field.
- **Size:** $15 * 8 \text{ bytes} + 2 * 4 \text{ bytes} = 128 \text{ bytes}$ exactly — $32 * \text{sizeof}(\text{uint32_t})$, matching `WARP_WIDTH`. This equality is asserted at construction time (§1.4).
- **`__align__(32)`:** a CUDA alignment attribute requesting 32-byte alignment (not to be confused with the 32-word size) — this keeps each slab’s start address friendly for coalesced/aligned memory transactions.
- **Who creates it:** never directly — the array `GpuSlabHash::d_table` is `cudaMalloc`’d as raw `int8_t*` and `reinterpret_cast` to `concurrent_slab<KeyT, ValueT>*` in `GpuSlabHashContext::initParameters`; overflow slabs are never “constructed” as C++ objects either — they are just 128-byte regions inside the allocator’s flat `uint32_t` array, accessed through raw pointer arithmetic (`getPointerFromSlab`).
- **Who owns its memory:** see §3.5 — base slabs are owned by `GpuSlabHash` (`d_table`), overflow slabs by `SlabAllocLight` (`d_super_blocks`).
- **Who accesses it:** exclusively `__device__` code, one warp at a time, through `GpuSlabHashContext::getPointerFromBucket` / `getPointerFromSlab`, which both return a `uint32_t*` (not a `concurrent_slab*!`) — the struct definition exists mainly to document the layout and drive the `sizeof(...)` assert; the actual access pattern treats a slab as 32 raw words indexed by `laneId` (see §11).
- **Exists on:** GPU only.

Visual layout of a `concurrent_slab`

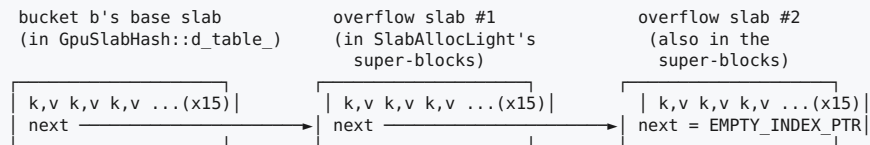
concurrent_slab<KeyT,ValueT> (128 bytes = 32 x uint32_t, one word per warp lane)

lane: 0 1 2 3 4 5 ... 26 27 28 29 30 31



next (lane 31) = SlabAddressT of the following slab, or
EMPTY_INDEX_POINTER (0xFFFFFFFF) if this is the last slab

Chained together, a bucket looks like:



The **REGULAR_NODE_KEY_MASK** constant that identifies which of the 32 lanes hold keys is a direct, verifiable consequence of this layout. For **ConcurrentMapT**:

```
// src/slab_hash_global.cuh
static constexpr uint32_t REGULAR_NODE_KEY_MASK = 0x15555555;
```

0x15555555 in binary is 1 0101 0101 0101 0101 0101 0101 0101, i.e. bits 0, 2, 4, 6, ..., 28 are set — **exactly the 15 even-numbered lanes** that hold **k0, k1, ..., k14** in the layout above. Every warp-vote that needs to check “does any key lane match?” ANDs the raw 32-bit ballot result with this mask so that value lanes (odd) and the next-pointer lane (31) can never be mistaken for a key match (see §7.3, §12).

4.3 key_only_slab<KeyT> and phase_concurrent_slab<KeyT, ValueT> — the other two layouts

```
// src/slab_hash_global.cuh
// this slab structure is meant to be used in either concurrent sets,
// or phase-concurrent maps.
// | key 0 | key 1 | key 2 | ... | key 30 | next_ptr |
template <typename KeyT>
struct __align__(32) key_only_slab {
    static constexpr uint32_t NUM_ELEMENTS_PER_SLAB = 31u;
    KeyT keys[NUM_ELEMENTS_PER_SLAB];
    uint32_t next_ptr_index[1];
};

template <typename KeyT, typename ValueT>
struct __align__(32) phase_concurrent_slab {
    static constexpr uint32_t NUM_ELEMENTS_PER_SLAB = 31u;
    // main slab (128 bytes), contain keys
    key_only_slab<KeyT> keys;

    // value storage:
    ValueT values[NUM_ELEMENTS_PER_SLAB];
};
```

- **key_only_slab<KeyT>** is the layout actually used by the **ConcurrentSet** variant (§9.2): 31 keys (no values, since a set only needs to know if a key is present) + 1 trailing “next” word = 32 words = 128 bytes. This is why **ConcurrentSetT::REGULAR_NODE_KEY_MASK** is **0x7FFFFFFF** (bits 0–30 set, i.e. lanes 0 through 30 all hold keys) and **ConcurrentSetT::NEXT_PTR_LANE** is explicitly **31u**.
- **phase_concurrent_slab<KeyT, ValueT>** bundles a **key_only_slab<KeyT>** (128 bytes of keys+next-pointer) together with a separate 31-element **values[]** array. **This struct is defined but this document could not find any `GpuSlabHash` / `GpuSlabHashContext` specialization, kernel, or warp operation anywhere in the repository that actually uses `phase_concurrent_slab`** (a full-repository search for the type name turns up only this one struct definition). The **SlabHashTypeT::PhaseConcurrentMap** enumerator and the matching **PhaseConcurrentMapT** traits class exist (`src/slab_hash_global.cuh`), but there is no **`GpuSlabHash<KeyT, ValueT, SlabHashTypeT::PhaseConcurrentMap>`** specialization anywhere — only the generic forward declaration `template <...> class GpuSlabHash;` is ever instantiated for **ConcurrentMap** and **ConcurrentSet**. This is treated in detail in §9.3; the short version is that **PhaseConcurrentMap is a stub / placeholder for future work**, not a working variant, and this document does not fabricate behavior for it.

4.4 SlabHashTypeT and the traits classes

```
// src/slab_hash_global.cuh
enum class SlabHashTypeT { ConcurrentMap, ConcurrentSet, PhaseConcurrentMap };

template <typename KeyT, typename ValueT>
class ConcurrentMapT {
public:
    static constexpr uint32_t A_INDEX_POINTER = 0xFFFFFFFF;
    static constexpr uint32_t EMPTY_INDEX_POINTER = 0xFFFFFFFF;
    static constexpr uint32_t BASE_UNIT_SIZE = 32;
    static constexpr uint32_t REGULAR_NODE_ADDRESS_MASK = 0x30000000;
    static constexpr uint32_t REGULAR_NODE_DATA_MASK = 0x3FFFFFFF;
    static constexpr uint32_t REGULAR_NODE_KEY_MASK = 0x15555555;

    using SlabTypeT = concurrent_slab<KeyT, ValueT>;

    static std::string getTypeName() { return std::string("ConcurrentMap"); }
};
```

`SlabHashTypeT` is a C++11 scoped enum used purely as a **template parameter/tag** — it selects which partial specialization of `GpuSlabHash` and `GpuSlabHashContext` gets instantiated (see §3.4 and §9). `ConcurrentMapT` (and its sibling `ConcurrentSetT`) is a small traits/policy class: it doesn't hold any data, it just bundles together compile-time constants (`EMPTY_INDEX_POINTER`, the masks, `BASE_UNIT_SIZE = 32` = one slab's word count) and a `using SlabTypeT = ...` alias that tells the rest of the code which struct from §4.2/§4.3 to use for this variant's slabs.

Two of `ConcurrentMapT`'s constants deserve a note: this document could not find any use of `REGULAR_NODE_ADDRESS_MASK` (`0x30000000`) or `REGULAR_NODE_DATA_MASK` (`0x3FFFFFFF`) anywhere in `src/` — like `ptr_index[0]` in §4.2, they appear to be unused, and are called out rather than assigned an invented purpose.

`A_INDEX_POINTER = 0xFFFFFFFF` is, by contrast, used constantly (§7) — it is a **sentinel value for the local variable** `next` inside every insert/search/delete/count loop, meaning “we are currently looking at the bucket's own base slab, not a chained slab yet.” It is deliberately different from `EMPTY_INDEX_POINTER = 0xFFFFFFFF`, which means “there is no next slab.”

4.5 SlabAllocLightContext — the allocator's per-warp state

Fully covered mechanically in §5, but summarized here for completeness as a core data structure:

```
// SlabAlloc/src/slab_alloc.cuh (private members of SlabAllocLightContext)
uint32_t* d_super_blocks; // pointer to ALL super-blocks (shared, not owned)
uint32_t hash_coef; // random coefficient for choosing a resident block
uint32_t num_attempts; // how many times this warp has had to rehash
uint32_t resident_index; // which memory block within a super-block is "home"
uint32_t resident_bitmap; // this warp's cached copy of its resident block's bitmap
uint32_t super_block_index; // which super-block is "home"
uint32_t allocated_index; // (declared; used transiently during warpAllocate)
```

- **Represents:** the allocator state that a *single warp* keeps in registers while it is actively allocating slabs.
- **Who creates it:** `SlabAllocLight`'s constructor creates the one “canonical” instance (`slab_alloc_context_`); every kernel that wants to allocate then makes its own **local copy** via the copy constructor (see `AllocatorContextT local_allocator_ctx(slab_hash.getAllocatorContext());` in `build.cuh`) and calls `initAllocator(tid, laneId)` on that local copy to give it a fresh per-warp resident block.
- **Who owns its memory:** it owns none — `d_super_blocks` is a non-owning copy of the pointer `SlabAllocLight` allocated.
- **Exists on:** both host and device — it's a `__host__ __device__` class, constructed on the host (inside `SlabAllocLight`) but *used* almost entirely on the device (`warpAllocate`, `freeUntouched`, etc. are all `__device__`-only).

5. The Slab Allocator, Deeply

5.1 Where the allocator is defined

The allocator is **not part of the SlabHash repository itself** — it lives in the `SlabAlloc` submodule (§2.2), specifically in `SlabAlloc/src/slab_alloc.cuh`. `SlabAlloc/src/slab_alloc.cu` is a one-line file (`#include "slab_alloc.cuh"`) that exists only so the allocator can also be compiled as a standalone translation unit for `SlabAlloc`'s own test target; `SlabHash` itself only ever includes the header. `SlabAlloc/src/slab_alloc_global.cuh` supplies just two things used throughout: the `SlabAllocAddressT` typedef (`using SlabAllocAddressT = uint32_t;`) and a `CHECK_ERROR` macro for CUDA error checking.

There are two classes, following the same owner/context split as §3:

- `SlabAllocLight<LOG_NUM_MEM_BLOCKS_, NUM_SUPER_BLOCKS_ALLOCATOR_, MEM_UNIT_WARP_MULTIPLES_>` — the owning class. `cudaMalloc`s the pool, initializes it, frees it in its destructor.
- `SlabAllocLightContext<...>` — the same three template parameters, a lightweight, copyable view with no ownership, meant to be embedded inside `GpuSlabHashContext` and copied into kernels.

In `SlabHash`, these are instantiated with fixed parameters chosen in `src/slab_hash_global.cuh`:

```
// src/slab_hash_global.cuh
namespace slab_alloc_par {
constexpr uint32_t log_num_mem_blocks = 8;
constexpr uint32_t num_super_blocks = 32;
constexpr uint32_t num_replicas = 1;
} // namespace slab_alloc_par

using DynamicAllocatorT = SlabAllocLight<slab_alloc_par::log_num_mem_blocks,
                                         slab_alloc_par::num_super_blocks,
                                         slab_alloc_par::num_replicas>;

using AllocatorContextT = SlabAllocLightContext<slab_alloc_par::log_num_mem_blocks,
                                                slab_alloc_par::num_super_blocks,
                                                slab_alloc_par::num_replicas>;
```

So every `GpuSlabHash` in this codebase uses an allocator with $2^8 = 256$ memory blocks per super-block, 32 super-blocks total, and a “memory unit” that is 1 warp-width (32 words = 128 bytes) — i.e. one memory unit is exactly one slab.

5.2 The three-level memory hierarchy: super-blocks, memory blocks, memory units

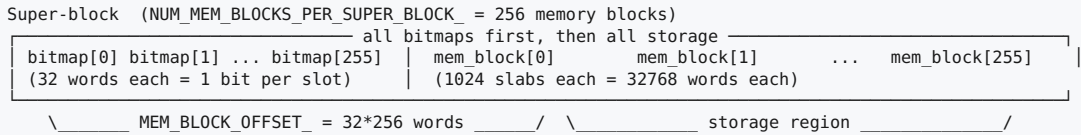
The allocator organizes its entire GPU memory pool as a fixed, three-level hierarchy, entirely described by `constexpr` values at the top of `SlabAllocLightContext`:

```
// SlabAlloc/src/slab_alloc.cuh
static constexpr uint32_t NUM_MEM_UNITS_PER_BLOCK_ = 1024;
static constexpr uint32_t NUM_BITMAP_PER_MEM_BLOCK_ = 32;
static constexpr uint32_t BITMAP_SIZE_ = 32;
static constexpr uint32_t WARP_SIZE_ = 32;
static constexpr uint32_t MEM_UNIT_SIZE_ = MEM_UNIT_WARP_MULTIPLES_ * WARP_SIZE_;
static constexpr uint32_t SUPER_BLOCK_BIT_OFFSET_ALLOC_ = 27;
static constexpr uint32_t MEM_BLOCK_BIT_OFFSET_ALLOC_ = 10;
static constexpr uint32_t MEM_UNIT_BIT_OFFSET_ALLOC_ = 5;
static constexpr uint32_t NUM_MEM_BLOCKS_PER_SUPER_BLOCK_ = (1 << LOG_NUM_MEM_BLOCKS_);
static constexpr uint32_t MEM_BLOCK_SIZE_ = NUM_MEM_UNITS_PER_BLOCK_ * MEM_UNIT_SIZE_;
static constexpr uint32_t SUPER_BLOCK_SIZE_ =
    ((BITMAP_SIZE_ + MEM_BLOCK_SIZE_) * NUM_MEM_BLOCKS_PER_SUPER_BLOCK_);
static constexpr uint32_t MEM_BLOCK_OFFSET_ =
    (BITMAP_SIZE_ * NUM_MEM_BLOCKS_PER_SUPER_BLOCK_);
```

With `SlabHash`'s chosen parameters (`LOG_NUM_MEM_BLOCKS_ = 8`, `num_super_blocks = 32`, `MEM_UNIT_WARP_MULTIPLES_ = 1`):

- **Memory unit** = 1 slab = 32 words = 128 bytes (`MEM_UNIT_SIZE_ = 32`).
- **Memory block** = 1024 memory units (`NUM_MEM_UNITS_PER_BLOCK_ = 1024`) = 4,096 slabs' worth of *bitmap coverage*, but note the memory block's *storage* area is `MEM_BLOCK_SIZE_ = 1024 * 32 = 32768` words, i.e. exactly 1024 slabs.
- Each memory block is preceded by its own **32-word bitmap** (`BITMAP_SIZE_ = 32`), one bit per memory unit — 32 words × 32 bits/word = 1024 bits = 1024 memory units tracked, matching `NUM_MEM_UNITS_PER_BLOCK_`.
- **Super-block** = $2^8 = 256$ memory blocks (`NUM_MEM_BLOCKS_PER_SUPER_BLOCK_ = 256`), so `SUPER_BLOCK_SIZE_ = (32 + 32768) * 256` words.
- **32 super-blocks total** — this is the full GPU allocation, one array, `d_super_blocks_`.

Visually, one super-block:



i.e. all the bitmaps for the entire super-block are stored *first*, as one contiguous region (`MEM_BLOCK_OFFSET`), and then all the memory blocks’ actual slab storage follows. This means “reading the bitmap for memory block k ” and “reading slab j inside memory block k ” touch very different regions of the array; the constant-time formulas that connect a compact 32-bit **address** to a real byte offset are exactly `getSuperBlockIndex`, `getMemBlockIndex`, `getMemBlockAddress`, `getMemUnitIndex`, and `addressDecoder` (§5.4).

5.3 The 32-bit allocator address format

Every allocated slab is identified by a single opaque `SlabAllocAddressT` (`uint32_t`) — this is the “address” that gets stored in a slab’s “next” word and passed around as `SlabAddressT`/`BucketAddressT` throughout SlabHash. `warpAllocate`’s own comment documents its bit layout precisely:

```
// SlabAlloc/src/slab_alloc.cuh, comment inside warpAllocate()
// allocated result: 5 bits: super_block index
//                  17 bits: memory block index
//                  5 bits: memory unit index (hi-bits of 10bit)
//                  5 bits: memory unit index (lo-bits of 10bit)
```

and it is assembled with exactly those bit offsets:

```
allocated_result = (super_block_index_ << SUPER_BLOCK_BIT_OFFSET_ALLOC_) |
                   (resident_index_ << MEM_BLOCK_BIT_OFFSET_ALLOC_) |
                   (laneId << MEM_UNIT_BIT_OFFSET_ALLOC_) | empty_lane;
```

with `SUPER_BLOCK_BIT_OFFSET_ALLOC_ = 27`, `MEM_BLOCK_BIT_OFFSET_ALLOC_ = 10`, `MEM_UNIT_BIT_OFFSET_ALLOC_ = 5`. So bits [31:27] = super-block index, bits [26:10] = memory block index (17 bits, though only `LOG_NUM_MEM_BLOCKS_ = 8` of those are meaningful given only 256 blocks per super-block), bits [9:5] = `laneId` (which lane’s bitmap word had the free bit), and bits [4:0] = `empty_lane` (which bit inside that word was free). Together, `laneId` (5 bits) and `empty_lane` (5 bits) form the 10-bit memory-unit index within the memory block (1024 units, matching `NUM_MEM_UNITS_PER_BLOCK_`).

5.4 Turning an address into a real GPU pointer

```
// SlabAlloc/src/slab_alloc.cuh
__host__ __device__ __forceinline__ SlabAllocAddressT
addressDecoder(SlabAllocAddressT address_ptr_index) {
    return getSuperBlockIndex(address_ptr_index) * SUPER_BLOCK_SIZE_ +
           getMemBlockAddress(address_ptr_index) +
           getMemUnitIndex(address_ptr_index) * MEM_UNIT_WARP_MULTIPLES_ * WARP_SIZE_;
}

__device__ __forceinline__ uint32_t* getPointerFromSlab(const SlabAllocAddressT& next,
                                                         const uint32_t& laneId) {
    return reinterpret_cast<uint32_t*>(d_super_blocks_) + addressDecoder(next) + laneId;
}
```

`addressDecoder` converts the compact 32-bit address into a **word offset** into the flat `d_super_blocks_` array: which super-block’s region to start in, plus that super-block’s `MEM_BLOCK_OFFSET`-adjusted memory-block storage address (`getMemBlockAddress`, which already skips past all the bitmaps — see §5.2’s diagram), plus the memory unit’s own word offset within that block. `getPointerFromSlab` then adds `laneId` on top — this is the single line that turns “slab address + which of the 32 lanes I am” into “the exact `uint32_t` this lane should read or write,” and it is the same function `GpuSlabHashContext::getPointerFromSlab` calls (§3.6) whenever any insert/search/delete/count operation needs to touch a chained (non-base) slab.

5.5 How a GPU warp obtains a slab: `warpAllocate`

This is the core of the allocator, and it is worth reading closely because its logic is the answer to “how does concurrent allocation work” (§5.7).

```

// SlabAlloc/src/slab_alloc.cuh
__device__ __forceinline__ uint32_t warpAllocate(const uint32_t& laneId) {
    int empty_lane = -1;
    uint32_t free_lane;
    uint32_t read_bitmap = resident_bitmap_;
    uint32_t allocated_result = 0xFFFFFFFF;

    while (allocated_result == 0xFFFFFFFF) {
        empty_lane = __ffs(~resident_bitmap_) - 1;
        free_lane = __ballot_sync(0xFFFFFFFF, empty_lane >= 0);
        if (free_lane == 0) {
            // all bitmaps are full: need to be rehashed again:
            updateMemBlockIndex((threadIdx.x + blockIdx.x * blockDim.x) >> 5);
            read_bitmap = resident_bitmap_;
            continue;
        }
        uint32_t src_lane = __ffs(free_lane) - 1;
        if (src_lane == laneId) {
            read_bitmap = atomicCAS(d_super_blocks_ + super_block_index_ * SUPER_BLOCK_SIZE_ +
                                   resident_index_ * BITMAP_SIZE_ + laneId,
                                   resident_bitmap_,
                                   resident_bitmap_ | (1 << empty_lane));
            if (read_bitmap == resident_bitmap_) {
                // successful attempt:
                resident_bitmap_ |= (1 << empty_lane);
                allocated_result = (super_block_index_ << SUPER_BLOCK_BIT_OFFSET_ALLOC_) |
                                   (resident_index_ << MEM_BLOCK_BIT_OFFSET_ALLOC_) |
                                   (laneId << MEM_UNIT_BIT_OFFSET_ALLOC_) | empty_lane;
            } else {
                // Not successful: updating the current bitmap
                resident_bitmap_ = read_bitmap;
            }
        }
        // asking for the allocated result;
        allocated_result = __shfl_sync(0xFFFFFFFF, allocated_result, src_lane);
    }
    return allocated_result;
}

```

Walking through it, remembering the key precondition — this is called by **one lane** of a warp on behalf of the whole warp (§5.7 explains why every lane still needs to call it):

1. **Each lane already has its own cached bitmap word, `resident_bitmap_`** — this is a per-lane register, loaded once per “resident block” selection (`initAllocator`/`updateMemBlockIndex`, §5.6). Since the whole warp’s 32 lanes together cover a 32-word bitmap (§5.2’s “32 words = 1024 bits”), each lane is one word of the current resident memory block’s bitmap.
2. `empty_lane = __ffs(~resident_bitmap_) - 1`: each lane inverts its own bitmap word and finds the first (least-significant) zero bit — i.e. the first *free* memory unit its word represents. `__ffs` returns a 1-based index (0 means “no bits set”), hence the `- 1`; `empty_lane == -1` means this lane’s 32-unit range is completely full.
3. `free_lane = __ballot_sync(0xFFFFFFFF, empty_lane >= 0)`: the whole warp votes — this produces a 32-bit mask of *which lanes* have at least one free slot. If it’s 0, **no lane in the entire resident memory block has a free unit**, so the warp calls `updateMemBlockIndex` to rehash to a different memory block (a new `resident_index_`/`super_block_index_`, chosen by perturbing the hash with `num_attempts_`, see §5.6) and retries the loop.
4. Otherwise, `src_lane = __ffs(free_lane) - 1` picks the **first lane (by lane index) that has a free slot** — this is the lane the whole warp will attempt to allocate from, this iteration.
5. Only that one lane (`if (src_lane == laneId)`) does the actual work: an `atomicCAS` on its bitmap word in global memory, trying to set the `empty_lane`-th bit — i.e. claim that specific memory unit. If the CAS succeeds (`read_bitmap == resident_bitmap_`, meaning nobody else changed the word between this lane’s last read and now), it assembles the final address (§5.3’s bit layout) into `allocated_result`. If it fails (someone else claimed a bit in the meantime, e.g. another warp resident in the same block), it just updates its local cache (`resident_bitmap_ = read_bitmap`) and the loop will try again (possibly picking a different `empty_lane` for the same `src_lane`, or moving to a different `src_lane` if that lane is now full).
6. `allocated_result = __shfl_sync(0xFFFFFFFF, allocated_result, src_lane)`: **every** lane in the warp reads back the result from `src_lane` via warp shuffle — so even though only one lane did the CAS, all 32 lanes leave the function with the same `allocated_result`, and the `while` loop condition is evaluated identically by every lane (this is required for `__ballot_sync`/`__shfl_sync` correctness — see §12).

The net effect: **a whole warp calls `warpAllocate` together, but only one slab is produced by the call**, chosen by a warp-wide vote for “which lane currently has a free bit,” followed by a single-lane `atomicCAS`.

5.6 Choosing (and re-choosing) a resident memory block

Before a warp can call `warpAllocate`, it needs a “resident” memory block — i.e., it needs to know *which* of the many bitmaps in `d_super_blocks_` its 32 lanes will treat as “their” bitmap this round:

```

// SlabAlloc/src/slab_alloc.cuh
// called at the beginning of the kernel:
__device__ __forceinline__ void createMemBlockIndex(uint32_t global_warp_id) {
    super_block_index_ = global_warp_id % num_super_blocks_;
    resident_index_ = (hash_coef_ * global_warp_id) >> (32 - LOG_NUM_MEM_BLOCKS_);
}

// called when the allocator fails to find an empty unit to allocate:
__device__ __forceinline__ void updateMemBlockIndex(uint32_t global_warp_id) {
    num_attempts_++;
    super_block_index_++;
    super_block_index_ =
        (super_block_index_ == num_super_blocks_) ? 0 : super_block_index_;
    resident_index_ =
        (hash_coef_ * (global_warp_id + num_attempts_)) >> (32 - LOG_NUM_MEM_BLOCKS_);
    // loading the assigned memory block:
    resident_bitmap_ = *((d_super_blocks_ + super_block_index_ * SUPER_BLOCK_SIZE_ +
        resident_index_ * BITMAP_SIZE_ + (threadIdx.x & 0x1f)));
}

// Objective: each warp selects its own resident warp allocator:
__device__ __forceinline__ void initAllocator(uint32_t& tid, uint32_t& laneId) {
    // hashing the memory block to be used:
    createMemBlockIndex(tid >> 5);

    // loading the assigned memory block:
    resident_bitmap_ = *((d_super_blocks_ + super_block_index_ * SUPER_BLOCK_SIZE_ +
        resident_index_ * BITMAP_SIZE_ + laneId);
    allocated_index_ = 0xFFFFFFFF;
}

```

`initAllocator` is called once per kernel/warp (visible in every kernel, e.g. `local_allocator_ctx.initAllocator(tid, laneId);` in `build.cuh`). `tid >> 5` is the **global warp id** (§7.1 explains this shift). The chosen `super_block_index_` is simply `global_warp_id % num_super_blocks_` — a round-robin spread of warps across the 32 super-blocks — and `resident_index_` is a hashed value derived from `global_warp_id` and a per-`SlabAllocLight`-instance random `hash_coef_` (set once, in `SlabAllocLight`'s constructor, from `std::mt19937 rng(time(0)); hash_coef_ = rng();`), right-shifted to keep only `LOG_NUM_MEM_BLOCKS_` (8) bits — i.e. a value in `[0, 256)`, matching `NUM_MEM_BLOCKS_PER_SUPER_BLOCK_`. This spreads different warps across different memory blocks so that not every warp contends for the same 32-word bitmap.

If a warp's resident block turns out to be completely full (`free_lane == 0` in `warpAllocate`), `updateMemBlockIndex` moves the warp to a new block: it advances `super_block_index_` by one (wrapping around), and rehashes `resident_index_` using `global_warp_id + num_attempts_` — perturbing the hash input with the retry count so that a warp that keeps failing doesn't keep landing on the same overfull block.

5.7 Why every lane calls `warpAllocate`, even though only one slab comes out

This is easy to misread as “one thread in the warp allocates for everyone.” It is actually the opposite: **`warpAllocate` must be called by all 32 threads of the warp together**, because it is built entirely out of warp-collective intrinsics (`__ffs` is per-thread, but `__ballot_sync` and `__shfl_sync` require every participating lane in the mask to call them, or behavior is undefined). Every caller in the repository respects this — e.g. in `insert.cuh`:

```

// only ONE lane's condition triggers the call, but every lane still
// executes allocateSlab() -> warpAllocate() because of how the branch
// is written: only laneId == 31 does the atomicCAS to LINK the new slab
// in, but *all* lanes call allocateSlab() first:
uint32_t new_node_ptr = allocateSlab(local_allocator_ctx, laneId);

if (laneId == 31) {
    ...
    uint32_t temp = atomicCAS((unsigned int*)p, SlabHashT::EMPTY_INDEX_POINTER, new_node_ptr);
    if (temp != SlabHashT::EMPTY_INDEX_POINTER) {
        freeSlab(new_node_ptr);
    }
}

```

Notice this line is **not** inside an `if (laneId == ...)` guard — `allocateSlab(...)` (and therefore `warpAllocate`) is called unconditionally by every lane whenever the warp collectively decides a new node is needed (i.e., this code path is only reached when the whole warp has already agreed, via the outer `work_queue` ballot loop of §7, that this is the situation). Only *after* the slab address comes back does the code narrow down to `laneId == 31` to perform the actual link-in `atomicCAS`. If that CAS loses a race (another warp already linked a different new slab in first, detected because `temp != EMPTY_INDEX_POINTER`), the just-allocated slab is freed again via `freeSlab` (§5.8) — it was allocated speculatively and turned out to be unnecessary.

5.8 Freeing a slab: `freeUntouched`

```
// SlabAlloc/src/slab_alloc.cuh
/*
This function, frees a recently allocated memory unit by a single thread.
Since it is untouched, there shouldn't be any worries for the actual memory
contents to be reset again.
*/
__device__ __forceinline__ void freeUntouched(SlabAllocAddressT ptr) {
    atomicAnd(d_super_blocks_ + getSuperBlockIndex(ptr) * SUPER_BLOCK_SIZE_ +
              getMemBlockIndex(ptr) * BITMAP_SIZE_ + (getMemUnitIndex(ptr) >> 5),
              ~(1 << (getMemUnitIndex(ptr) & 0x1F)));
}
```

The name and comment are precise: this only frees a slab that was **just allocated and never written to** — i.e., the losing side of the race in §5.7, where a warp allocated a node speculatively but another warp’s CAS won first. It is a single-thread operation (called only from `laneId == 31`) that clears the corresponding bit in the owning memory block’s bitmap with an `atomicAnd`. **There is no general-purpose “free this slab that holds live key/value data” path anywhere in the repository** — deletion (§6, §9) works by overwriting a slab’s *contents* (`EMPTY_PAIR_64`) in place, not by returning the slab to the allocator. This means a slab, once linked into a chain, is never released back to the pool for reuse by a different bucket, even after every key inside it has been deleted — a design trade-off the code itself does not comment on further; this document does not speculate about future plans beyond what §5.8’s docstring and the absence of any other `free`-style function establish.

5.9 Bulk allocation: `warpAllocateBulk`

The allocator also exposes a bulk variant, used for allocating k consecutive units from the same lane’s bitmap word in one shot:

```
// SlabAlloc/src/slab_alloc.cuh
__device__ __forceinline__ uint32_t warpAllocateBulk(uint32_t& laneId,
                                                    const uint32_t k) {
    ...
    empty_lane =
        32 -
        (__ffs(__brev(~resident_bitmap_))); // reversing the order of assigning lanes
                                           // compared to single allocations
    const uint32_t mask = ((1 << k) - 1) << (empty_lane - k + 1);
    free_lane = __ballot_sync(
        0xFFFFFFFF,
        (empty_lane >= (k - 1)) &&
        !(resident_bitmap_ & mask));
    ...
}
```

Instead of finding any single free bit, this reverses the bit order (`__brev`) to search from the *high* end of the word and checks whether a contiguous run of k free bits exists starting there, building a k -bit `mask` to test and later CAS in one atomic operation. This document did not find any call site for `warpAllocateBulk` anywhere in `src/` (SlabHash proper) — it is available on the allocator’s public interface but unused by the current `ConcurrentMap/ConcurrentSet` code paths, which only ever call the single-unit `warpAllocate`.

5.10 How `GpuSlabHashContext` and `GpuSlabHash` interact with the allocator — recap

Already established in §3.6, restated here for completeness of this section: `GpuSlabHash`’s constructor obtains a pointer to the *canonical* allocator context via `dynamic_allocator_->getContextPtr()` and copies it into its own `gpu_context_` once, at construction time. From then on, every kernel that needs to allocate makes a **fresh local copy** of that context and calls `initAllocator` on it (§5.6) — this is why kernel bodies consistently show the pattern:

```
AllocatorContextT local_allocator_ctx(slab_hash.getAllocatorContext());
local_allocator_ctx.initAllocator(tid, laneId);
```

`slab_hash.getAllocatorContext()` returns a reference to `GpuSlabHashContext`’s own `global_allocator_ctx_` member (itself a value-copy made once at `GpuSlabHash` construction, per §3.2); the *local* copy is what actually receives a resident block for this kernel launch. Search-only kernels (which never allocate) skip this step entirely — e.g. `search_table` in `search_kernel.cuh` never touches the allocator context at all, because searching never needs to create new slabs.

6. The Hash-Table Implementation (`GpuSlabHash` , `ConcurrentMap`)

This section traces `GpuSlabHash<KeyT, ValueT, SlabHashTypeT::ConcurrentMap>` end-to-end. §9 covers how `ConcurrentSet` differs.

6.1 Initialization

```
// src/concurrent_map/cmap_class.cuh
GpuSlabHash(const uint32_t num_buckets,
            DynamicAllocatorT* dynamic_allocator,
            uint32_t device_idx,
            const time_t seed = 0,
            const bool identity_hash = false)
    : num_buckets_(num_buckets)
    , d_table_(nullptr)
    , slab_unit_size_(0)
    , dynamic_allocator_(dynamic_allocator)
    , device_idx_(device_idx) {
assert(dynamic_allocator && "No proper dynamic allocator attached to the slab hash.");
assert(sizeof(typename ConcurrentMapT<KeyT, ValueT>::SlabTypeT) ==
        (WARP_WIDTH * sizeof(uint32_t)) &&
        "A single slab on a ConcurrentMap should be 128 bytes");
int32_t devCount = 0;
CHECK_CUDA_ERROR(cudaGetDeviceCount(&devCount));
assert(device_idx_ < devCount);

CHECK_CUDA_ERROR(cudaSetDevice(device_idx_));

slab_unit_size_ =
    GpuSlabHashContext<KeyT, ValueT, SlabHashTypeT::ConcurrentMap>::getSlabUnitSize();

// allocating initial buckets:
CHECK_CUDA_ERROR(cudaMalloc((void*)&d_table_, slab_unit_size_ * num_buckets_));

CHECK_CUDA_ERROR(cudaMemset(d_table_, 0xFF, slab_unit_size_ * num_buckets_));

// creating a random number generator:
if (!identity_hash) {
    std::mt19937 rng(seed ? seed : time(0));
    hf_.x = rng() % PRIME_DIVISOR;
    if (hf_.x < 1)
        hf_.x = 1;
    hf_.y = rng() % PRIME_DIVISOR;
} else {
    hf_ = {0u, 0u};
}

// initializing the gpu_context_:
gpu_context_.initParameters(
    num_buckets_, hf_.x, hf_.y, d_table_, dynamic_allocator_->getContextPtr());
}
```

Step by step: (1) sanity checks — a real allocator must be attached, and the slab type really must be 128 bytes; (2) `cudaMalloc` **one base slab per bucket** into `d_table_`; (3) `cudaMemset(d_table_, 0xFF, ...)` — this single call is what makes an empty table’s every key word equal `EMPTY_KEY` (`0xFFFFFFFF`) and every next-pointer word equal `EMPTY_INDEX_POINTER` (`0xFFFFFFFF`) simultaneously, since both sentinels happen to be the same bit pattern; (4) generate a random 2-parameter hash function (`hf_.x`, `hf_.y`) via `std::mt19937`, unless `identity_hash` was requested (used by `test/iterator_test.cu` so bucket assignment is predictable); (5) copy everything the device needs — including a pointer to the *allocator’s* context, obtained via `dynamic_allocator_>getContextPtr()` — into `gpu_context_` via `initParameters` (§3.2 shows this function’s body).

6.2 Buckets: how they are created, and how a key maps to one

There is no separate “bucket” object or “create bucket” function — a bucket is simply index `i` into the `d_table_` array of base slabs, and all `num_buckets_` of them are created together, up front, by the single `cudaMalloc/cudaMemset` pair above. The number of buckets is fixed for the lifetime of a `GpuSlabHash` instance — **the table does not grow the number of buckets dynamically** (see §6.9 for exactly what *does* grow).

Mapping a key to a bucket is a single `__device__ __host__` function on `GpuSlabHashContext`:

```
// src/concurrent_map/cmap_class.cuh
static constexpr uint32_t PRIME_DIVISOR = 4294967291u;
...
__device__ __host__ forceinline uint32_t computeBucket(const KeyT& key) const {
    return (((hash_x ^ key) + hash_y) % PRIME_DIVISOR) % num_buckets_;
}
```

This is a standard multiplicative-ish universal hash: XOR the key with `hash_x_`, add `hash_y_`, reduce modulo the large prime `4294967291` (the largest prime below 2^{32}), then reduce modulo `num_buckets_` to land in range. `hash_x_ / hash_y_` are the `(hf_.x, hf_.y)` generated in §6.1 — the *same* two numbers for the whole table’s lifetime, meaning `computeBucket` is deterministic per table instance (a prerequisite for later being able to find a key you previously inserted).

6.3 Collisions and how slabs are attached to buckets

Collision handling is **separate chaining**: multiple keys mapping to the same bucket all live somewhere in that bucket’s slab chain (§3.5, §4.2). A new slab is attached to a chain in exactly one place in the codebase — the “no empty slot, no next pointer yet” branch inside `insertPair` / `insertPairUnique` / `insertKey` (§6.5) — always via a single-lane `atomicCAS` on the current tail slab’s lane-31 word, changing it from `EMPTY_INDEX_POINTER` to the newly allocated slab’s address. This is a classic **lock-free linked-list append**: the CAS guarantees that if two warps race to extend the same chain at the same time, only one of them successfully links its slab in; the other observes the CAS fail (the word is no longer `EMPTY_INDEX_POINTER`) and instead follows the link the winner just installed.

6.4 How keys/values are stored

Storing (once a destination slab and empty lane are known) is a **single 64-bit atomic compare-and-swap**, packing key and value into one 8-byte word:

```
// src/concurrent_map/warp/insert.cuh (inside insertPair)
old_key_value_pair =
    atomicCAS((unsigned long long int*)p,
              EMPTY_PAIR_64,
              ((uint64_t)(*reinterpret_cast<const uint32_t*>(
                  reinterpret_cast<const unsigned char*>(&myValue)))
               << 32) |
               *reinterpret_cast<const uint32_t*>(
                  reinterpret_cast<const unsigned char*>(&myKey))));
if (old_key_value_pair == EMPTY_PAIR_64)
    to_be_inserted = false; // successful insertion
```

`p` here is a `uint32_t*` pointing at the **key** word of the destination `key_value_pair` (an even-numbered lane, per §4.2’s layout); casting it to `unsigned long long int*` treats that word *and the very next word* (the value, guaranteed adjacent because `key_value_pair` packs **key** then **value**) as one 64-bit unit. The new 64-bit value is built by shifting the value into the high 32 bits and OR-ing the key into the low 32 bits — matching x86/GPU little-endian layout, where the low 32 bits of a 64-bit word are the lower address, i.e. the **key** field. `EMPTY_PAIR_64` (`0xFFFFFFFFFFFFFFFF`, from `slab_hash_global.cuh`) is both fields being `EMPTY_KEY` / `EMPTY_VALUE` simultaneously — exactly what the `cudaMemset(..., 0xFF, ...)` in §6.1 produces. The CAS only succeeds if the destination slot is still empty (nobody else raced in first); success is detected by comparing the CAS’s *old* return value against `EMPTY_PAIR_64`.

This single-instruction key+value write is why the implementation never suffers a “torn write” where another thread could observe a key with no matching value (or vice versa) — the whole pair becomes visible atomically.

6.5 Insert — the actual code path

Traced through the real call chain (`gpu_hash_table::hash_build` → `GpuSlabHash::buildBulk` → `build_table_kernel` → `GpuSlabHashContext::insertPair`):

```

gpu_hash_table::hash_build(h_key, h_value, num_keys) [src/gpu_hash_table.cuh]
  cudaMemcpy key/value host -> device
  slab_hash->buildBulk(d_key_, d_value_, num_keys)
    |
    v
GpuSlabHash::buildBulk(d_key, d_value, num_keys) [src/concurrent_map/cmap_implementation.cuh]
  computes num_blocks = ceil(num_keys / BLOCKSIZE_) (BLOCKSIZE_ = 128)
  build_table_kernel<<<num_blocks, BLOCKSIZE_>>>(d_key, d_value, num_keys, gpu_context_)
    |
    v
build_table_kernel(...) [src/concurrent_map/device/build.cuh]
  tid = threadIdx.x + blockIdx.x * blockDim.x
  laneId = threadIdx.x & 0x1F
  local_allocator_ctx = copy of slab_hash's allocator context; initAllocator(tid, laneId)
  if (tid < num_keys): myKey/myValue = d_key[tid]/d_value[tid]; myBucket = slab_hash.computeBucket(myKey)
  slab_hash.insertPair(to_insert, laneId, myKey, myValue, myBucket, local_allocator_ctx)
    |
    v
GpuSlabHashContext::insertPair(...) [src/concurrent_map/warp/insert.cuh]
  warp-cooperative ballot loop (see §7 for full mechanics):
  - vote on which lanes still have work (to_be_inserted)
  - read the current slab (base bucket slab, or wherever `next` points)
  - ballot for an EMPTY KEY lane
  - if found: single-lane atomicCAS of the 64-bit key/value pair (§6.4)
  - if not found and next == EMPTY_INDEX_POINTER: allocateSlab (§5.5) + atomicCAS link-in (§6.3)
  - if not found and next != EMPTY_INDEX_POINTER: advance `next`, loop again

```

There are two insert entry points, both defined in `src/concurrent_map/warp/insert.cuh` and both reachable from `GpuSlabHash`:

- **insertPair** (used by `buildBulk` → `build_table_kernel`): always inserts, even if the key already exists — duplicate keys are allowed to coexist as separate slots (this is why `count_key / countKey`, §6.8, needs to sum matches across the *whole* chain rather than stop at the first hit).
- **insertPairUnique** (used by `buildBulkWithUniqueKeys` → `build_table_with_unique_keys_kernel`): first checks, every time it reads a slab, whether the slab already contains this exact key (`isExisting = __ballot_sync(...) & REGULAR_NODE_KEY_MASK`); if so, it marks `to_be_inserted = false` without writing anything, so **only the first occurrence of a key is ever stored**. It returns `true` only if a genuinely new key was written (`new_insertion`).

6.6 Search — the actual code path

```

gpu_hash_table::hash_search(h_query, h_result, num_queries) [src/gpu_hash_table.cuh]
  cudaMemcpy queries host -> device; cudaMemcpy results to 0xFF
  slab_hash->searchIndividual(d_query_, d_result_, num_queries)
    |
    v
GpuSlabHash::searchIndividual(...) [src/concurrent_map/cmap_implementation.cuh]
  search_table<<<num_blocks, BLOCKSIZE_>>>(d_query, d_result, num_queries, gpu_context_)
    |
    v
search_table(...) [src/concurrent_map/device/search_kernel.cuh]
  tid, laneId computed the same way as build_table_kernel
  myQuery = d_queries[tid]; myBucket = slab_hash.computeBucket(myQuery)
  slab_hash.searchKey(to_search, laneId, myQuery, myResult, myBucket)
  d_results[tid] = myResult
    |
    v
GpuSlabHashContext::searchKey(...) [src/concurrent_map/warp/search.cuh]
  ballot loop (§7):
  - read current slab (base or chained, via `next`)
  - ballot for src_unit_data == wanted_key, masked to key lanes
  - if found: shuffle the ADJACENT lane's word (found_lane + 1 = the value) back to src_lane
  - if not found and next == EMPTY_INDEX_POINTER: myValue = SEARCH_NOT_FOUND
  - if not found and next != EMPTY_INDEX_POINTER: advance `next`, loop again

```

Note that `search_table` never initializes an allocator context — searching never allocates, so `getAllocatorContext() / initAllocator` are absent from this kernel (contrast with `build_table_kernel`, §6.5). A second search entry point, `searchKeyBulk` (called from `search_table_bulk / GpuSlabHash::searchBulk`), implements the *same* logic but without the outer warp-ballot “who still has work” loop — instead it always iterates over all 32 `src_lane` values in a fixed `#pragma unroll` loop, because “bulk” search assumes **every** lane in the warp always has a real query (no `to_be_searched` flag needed), which lets the compiler unroll the whole 32-iteration loop at compile time.

6.7 Delete — the actual code path

```

gpu_hash_table::hash_delete(h_key, num_keys)                                [src/gpu_hash_table.cuh]
  cudaMemcpy keys host -> device
  slab_hash->deleteIndividual(d_key_, num_keys)
    |
    v
GpuSlabHash::deleteIndividual(...)                                         [src/concurrent_map/cmap_implementation.cuh]
  delete_table_keys<<num_blocks, BLOCKSIZE_>>>(d_key, num_keys, gpu_context_)
    |
    v
delete_table_keys(...)                                                    [src/concurrent_map/device/delete_kernel.cuh]
  tid, laneId, myKey, myBucket computed the same way
  slab_hash.deleteKey(to_delete, laneId, myKey, myBucket)
    |
    v
GpuSlabHashContext::deleteKey(...)                                       [src/concurrent_map/warp/delete.cuh]
  ballot loop (§7):
  - read current slab (base or chained)
  - ballot for src_unit_data == src_key, masked to key lanes
  - if found: single-lane atomicExch of the 64-bit slot to EMPTY_PAIR_64 (not atomicCAS!)
    successful_deletion = (the OLD key we just overwrote == the key we wanted)
  - if not found and next == EMPTY_INDEX_POINTER: give up (key not present)
  - if not found and next != EMPTY_INDEX_POINTER: advance `next`, loop again

```

Delete uses `atomicExch`, not `atomicCAS`, on the 64-bit slot:

```

// src/concurrent_map/warp/delete.cuh
uint64_t old_pair = atomicExch((unsigned long long int*)p, EMPTY_PAIR_64);
uint32_t deleted_key = static_cast<uint32_t>(old_pair);
successful_deletion = deleted_key == src_key;

```

This unconditionally overwrites the slot and then checks *after the fact* whether the value it just clobbered was really the key it meant to delete. This is safe here (unlike a naive unconditional overwrite would be in general) precisely because the warp has *already* established, via the ballot in the previous step, that lane `dest_lane` currently holds `src_key` — the `atomicExch` is racing only against other threads that might be *reading* or also trying to delete the same slot concurrently, not against uncertainty about what’s currently there. Only the first matching instance found while walking the chain is deleted — `deleteKey`’s own comment says so directly: “delete the first instance of key”.

6.8 Count — a fourth operation, not requested by name but present in the code

Not explicitly one of insert/search/delete, but implemented alongside them and worth tracing because it demonstrates the same pattern with a twist — **it never stops at the first match**:

```

// src/concurrent_map/warp/count.cuh (inside the ballot loop)
const int wanted_key_count = __popc(__ballot_sync(0xFFFFFFFF, src_unit_data == wanted_key) &
    SlabHashT::REGULAR_NODE_KEY_MASK);

if(laneId == src_lane) //count
  myCount += wanted_key_count;

uint32_t next_ptr = __shfl_sync(0xFFFFFFFF, src_unit_data, 31, 32); //iterate
if (next_ptr == SlabHashT::EMPTY_INDEX_POINTER){
  if(laneId == src_lane){
    to_be_searched = false;
  }
}
else{
  next = next_ptr;
}

```

`__popc` (population count — count of set bits) on the masked ballot result counts **how many key lanes in this one slab matched**, and this is accumulated (`myCount +=`) across the *entire* chain — the loop only terminates when `next_ptr == EMPTY_INDEX_POINTER`, unlike `searchKey` / `deleteKey`, which stop as soon as one match is found. This makes `countKey` the operation that correctly handles the fact that `insertPair` (§6.5, not `insertPairUnique`) allows duplicate keys.

6.9 Does the table grow dynamically?

The number of buckets is fixed (§6.2) — `GpuSlabHash` never `cudaMalloc`s a larger `d_table_` or rehashes into more buckets. What *does* grow dynamically is the **length of each bucket’s chain**: as more keys land in a bucket than its base slab’s 15 slots can hold, `insertPair` allocates additional 128-byte slabs from the allocator (§5, §6.3) and links them onto the end of that bucket’s chain, one at a time, on demand. This is the sense in which the README and this document call SlabHash a “dynamic hash table” — dynamic *slab allocation for growing chains*, not dynamic *bucket-count* resizing. There is no code anywhere in the repository that changes `num_buckets_` after construction, and no “resize”/“rehash-to-more-buckets” function exists on `GpuSlabHash`.

7. Warp-Level Execution

7.1 `tid` and `laneId` — computed identically everywhere

Every kernel in the repository computes these two values the same way, in the same two lines, right at the top:

```
uint32_t tid = threadIdx.x + blockIdx.x * blockDim.x;
uint32_t laneId = threadIdx.x & 0x1F;
```

- `tid` is the **global thread index** — this thread’s position among *all* threads launched across *all* blocks in the grid. With `BLOCKSIZE = 128` (four warps per block, fixed in `GpuSlabHash`’s private constants), `tid` ranges from 0 to `num_blocks * 128 - 1`.
- `laneId = threadIdx.x & 0x1F` masks off everything but the low 5 bits of the *block-local* thread index — since a warp is exactly 32 threads and CUDA guarantees warps are contiguous, aligned 32-thread groups within a block, `threadIdx.x & 0x1F` is exactly this thread’s position within its own warp, a number from 0 to 31, **regardless of which warp or block it’s in**. This is what “lane” means throughout this document: not a global thread index, but a 0–31 position inside one warp.

`tid >> 5` (used by the allocator, §5.6, and by `bucket_count_kernel`, §6.8’s sibling in `misc_kernels.cuh`) is therefore the **global warp id** — dividing the global thread index by 32 discards the lane bits, leaving which warp (across the whole grid) this thread belongs to.

Every kernel also does an early-exit guard using these two values together:

```
if ((tid - laneId) >= num_keys) {
    return;
}
```

`tid - laneId` is the global index of **lane 0 of this thread’s warp** — i.e. this checks “is this entire warp beyond the valid range of work,” not “is this individual thread beyond it.” This is deliberate and important: even if an individual thread’s `tid >= num_keys` (so it personally has no key to process), **the thread must not return early** if any other lane in its warp still has real work, because the warp-cooperative operations below require *all 32 lanes* to participate in `__ballot_sync`/`__shfl_sync` calls, or the result is undefined. Threads with no real work instead fall through with `to_insert/to_search/to_delete` left `false`, so they still execute the ballot loop (contributing a `0` bit to every ballot) without ever having their own `myKey` processed.

7.2 The `work_queue` ballot loop — the core cooperative pattern

Every one of `insertPair`, `insertPairUnique`, `searchKey`, `countKey`, and `deleteKey` is built around the *same* loop skeleton. Here it is, extracted from `searchKey` (`src/concurrent_map/warp/search.cuh`) with only the comparison-specific lines omitted:

```
uint32_t work_queue = 0;
uint32_t last_work_queue = work_queue;
uint32_t next = SlabHashT::A_INDEX_POINTER;

while ((work_queue = __ballot_sync(0xFFFFFFFF, to_be_searched))) {
    next = (last_work_queue != work_queue) ? SlabHashT::A_INDEX_POINTER : next;
    uint32_t src_lane = __ffs(work_queue) - 1;
    uint32_t src_bucket = __shfl_sync(0xFFFFFFFF, bucket_id, src_lane, 32);
    uint32_t wanted_key = __shfl_sync(0xFFFFFFFF, /* myKey reinterpreted */, src_lane, 32);

    const uint32_t src_unit_data = (next == SlabHashT::A_INDEX_POINTER)
        ? *(getPointerFromBucket(src_bucket, laneId))
        : *(getPointerFromSlab(next, laneId));
    /* ... compare src_unit_data against wanted_key, act on the result ... */

    last_work_queue = work_queue;
}
```

This is a **warp-wide work queue drained one source lane at a time**:

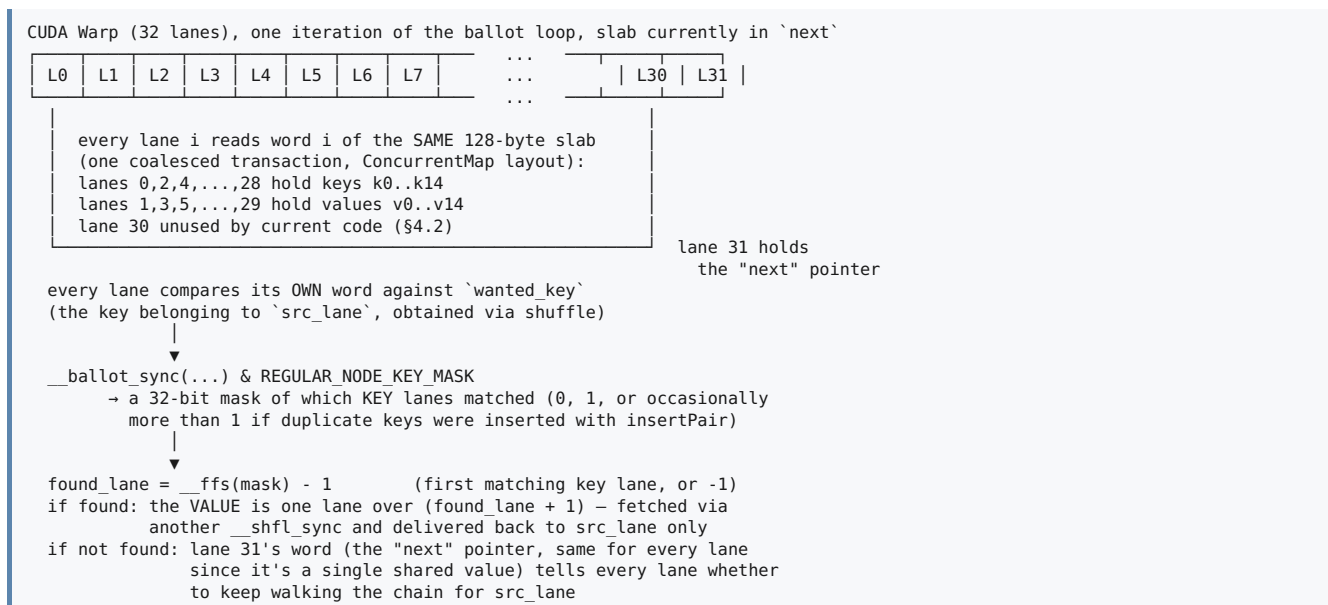
- `work_queue = __ballot_sync(0xFFFFFFFF, to_be_searched)`: every lane votes whether it *still* has unfinished work. The result is a 32-bit mask — bit *i* set means lane *i* still needs something done. The `while` loop exits only once **every** lane in the warp has finished (mask becomes 0).
- `next = (last_work_queue != work_queue) ? A_INDEX_POINTER : next`: this resets the traversal pointer to “start over at the bucket’s base slab” **whenever the work queue has changed since the last iteration** — i.e. whenever the *previous* iteration’s source lane finished (found its key, or gave up), the *next* source lane picked always starts its

own chain walk from the bucket base, not wherever the previous lane’s walk had gotten to. (If the work queue is unchanged, it means the same source lane is still being served — e.g. its key wasn’t in this slab, so `next` was just updated to the chain’s next-pointer and the loop continues *for that lane*.)

3. `src_lane = __ffs(work_queue) - 1`: picks the **lowest-numbered lane** that still has work — this becomes “the lane the whole warp works for, this iteration.”
4. `__shfl_sync(0xFFFFFFFF, bucket_id, src_lane, 32)` and the equivalent for the key: **every lane in the warp fetches `src_lane`’s bucket id and key** via warp shuffle — this is how one lane’s query becomes visible to all 32 lanes without touching shared or global memory.
5. `src_unit_data = *(getPointerFromBucket(...))` or `*(getPointerFromSlab(...))`: **every lane reads its own word** (indexed by its own `laneId`) of the *same* 128-byte slab — this is the coalesced, warp-wide 128-byte read that makes the whole scheme fast: 32 lanes issue 32 adjacent 4-byte loads that the memory system coalesces into one transaction.
6. The comparison/action logic (omitted above, detailed per-operation in §6) runs, potentially clearing `to_be_searched` for `src_lane` if it’s done.
7. `last_work_queue = work_queue` and the loop repeats — but note that **all 32 lanes participate in every iteration**, even though only `src_lane`’s query is actually being resolved this round. Lanes that already finished (`to_be_searched == false`) still execute the loop body (they just don’t contribute to future ballots), and lanes whose own query hasn’t come up yet keep fetching and discarding data for other lanes’ queries until their own turn arrives.

7.3 What each lane is doing — concretely

For a single iteration of the loop above, once `src_lane` has been chosen and a slab has been fetched:



Only `src_lane` ever writes a result (`myValue`, or clears `to_be_searched` for delete/search) — every other lane’s participation in this iteration is “read the slab, compute the same ballot, and discard everything except the next-pointer,” purely so the warp-collective intrinsics have a full, well-defined 32-lane mask to operate on. This is the literal answer to “why use a warp instead of independent threads”: **the slab is 32 words, and the hardware only offers a single coalesced 128-byte transaction if all 32 lanes participate — so the implementation makes all 32 lanes participate in every slab read, one query at a time, rather than have 32 independent threads issue 32 separate uncoalesced reads for 32 different queries.**

7.4 Synchronization primitives used, and why

- `__ballot_sync(0xFFFFFFFF, predicate)`: used (a) to build the `work_queue` (which lanes still have work), and (b) to find which key lane matches inside a fetched slab. The `0xFFFFFFFF` mask means “all 32 lanes participate” — required because every lane does reach this call (per §7.1’s early-exit design).
- `__shfl_sync(0xFFFFFFFF, value, src_lane, 32)`: used to broadcast `src_lane`’s bucket id, key, and (on the value side) the matched value’s word, from one lane to all 32 — the mechanism by which one lane’s private register data becomes visible to the whole warp without any memory traffic.
- `__ffs(x)`: find-first-set-bit (1-based, returns 0 if `x == 0`) — used both to pick `src_lane` from the `work_queue` mask and to pick which lane inside a ballot result is the “found” or “empty” lane.
- `__popc(x)`: population count, used only in `countKey` (§6.8) to count *how many* key lanes matched, not just whether one did.

- `atomicCAS` (32-bit, for linking a new slab's pointer in; 64-bit, for inserting a key/value pair) and `atomicExch` (64-bit, for deleting a pair) and `atomicAnd` (in the allocator, for freeing a bitmap bit): the only points where the warp-cooperative logic above hands off to genuine cross-warp/cross-thread races — covered in full in §12.

7.5 Why a warp, not independent threads — summarized

1. A slab is deliberately sized to exactly match a warp (128 bytes = 32 lanes × 4 bytes), so reading/writing one slab as one coalesced transaction requires all 32 lanes to cooperate (§7.3).
2. `__ballot_sync`/`__shfl_sync` are hardware warp intrinsics — there is no equivalent efficient primitive at the granularity of “a handful of independent threads,” so any cooperative scheme naturally aligns to the warp.
3. The slab allocator itself is warp-synchronized (`warpAllocate`, §5.5) — its whole design (one lane's bitmap word per lane) only makes sense if the 32 lanes calling it correspond to the 32 words of a bitmap, reinforcing the warp as the unit of cooperation end to end, from allocation through storage layout to traversal.

8. `GpuSlabHashContext` in Detail

8.1 Why it exists

The file comment above its definition states the reason directly:

```
// src/concurrent_map/cmap_class.cuh
/*
 * This is the main class that will be shallowly copied into the device to be
 * used at runtime. This class does not own the allocated memory on the gpu
 * (i.e., d_table_)
 */
```

`GpuSlabHash` cannot itself be passed into a `__global__` kernel launch the way it is written: it holds `dynamic_allocator_`, a pointer to a full `SlabAllocLight` host object that manages `cudaMalloc/cudaFree` — passing that whole object by value into device code would be both wasteful and meaningless (its constructors/destructors are host-oriented, and it carries far more state than a kernel needs). `GpuSlabHashContext` exists purely to be the **small, POD-like, trivially copyable subset of that state** — just enough for device code to compute a bucket, fetch a slab, and call the allocator — that is cheap to duplicate into every kernel launch’s parameter space.

8.2 How it differs from `GpuSlabHash`

	<code>GpuSlabHash</code>	<code>GpuSlabHashContext</code>
Constructor does <code>cudaMalloc/cudaFree</code>	Yes (<code>d_table_</code>)	No
Holds <code>dynamic_allocator_</code>	as a raw pointer to the host-owned <code>SlabAllocLight</code>	as a value copy of just its <code>AllocatorContextT</code>
Has a destructor that frees GPU memory	Yes (<code>~GpuSlabHash() { cudaFree(d_table_); }</code>)	No (<code>~GpuSlabHashContext() {}</code> — empty)
Exposes <code>buildBulk</code> , <code>searchIndividual</code> , etc. (kernel <i>launchers</i>)	Yes	No
Exposes <code>insertPair</code> , <code>searchKey</code> , etc. (the actual warp-cooperative logic, callable from <code>__device__</code> code)	No	Yes
Ever appears in a <code>__global__</code> kernel signature	No	Yes, always by value
Random hash-function generation (<code>std::mt19937</code>)	Yes, in its constructor	No — receives <code>hash_x_/hash_y_</code> as plain <code>uint32_t</code> s via <code>initParameters</code>

8.3 Members

```
// src/concurrent_map/cmap_class.cuh, private section of GpuSlabHashContext
uint32_t num_buckets_;
uint32_t hash_x_;
uint32_t hash_y_;
typename ConcurrentMapT<KeyT, ValueT>::SlabTypeT* d_table_;
// a copy of dynamic allocator's context to be used on the GPU
AllocatorContextT global_allocator_ctx_;
```

Five members, all either scalars or a single pointer/value-type, and the whole class has **no user-defined copy assignment operator or move semantics beyond the compiler-generated ones** — its custom copy *constructor* (below) exists mainly for symmetry/clarity, since a compiler-generated memberwise copy would do the same thing for these five POD-ish members.

8.4 Methods it exposes

Grouped by purpose:

- Construction/copying:** a default constructor (zero-initializes everything), a copy constructor that explicitly copies each member through its getter, and an empty destructor.
- Host-side setup:** `initParameters(...)` (§3.2 — called once, from `GpuSlabHash`’s constructor) and the `static` helpers `getSlabUnitSize()` (returns `sizeof(SlabTypeT)`, used by `GpuSlabHash` to size its `cudaMalloc`) and `getSlabHashTypeName()`.

- **Trivial accessors**, all `__device__ __host__`: `getAllocatorContext()`, `getDeviceTablePointer()`, `getNumBuckets()`, `getHashX()`, `getHashY()`.
- **The actual data-structure operations**, all `__device__`-only (and `__forceinline__`, meaning the compiler is instructed to inline them directly into the calling kernel rather than emit a real function call): `computeBucket`, `insertPair`, `insertPairUnique`, `searchKey`, `searchKeyBulk`, `countKey`, `deleteKey`.
- **Address-to-pointer helpers**, also `__device__`-only: `getPointerFromSlab` (delegates straight to the allocator context's identically named method, §5.4) and `getPointerFromBucket`:

```
__device__ __forceinline__ uint32_t* getPointerFromBucket(const uint32_t bucket_id,
                                                         const uint32_t laneId) {
    return reinterpret_cast<uint32_t*>(d_table_) +
        bucket_id * ConcurrentMapT<KeyT, ValueT>::BASE_UNIT_SIZE + laneId;
}
```

This is the base-slab equivalent of `getPointerFromSlab` — it does simple pointer arithmetic directly against `d_table_` (no allocator involved, no address decoding needed, since base slabs are stored contiguously and `bucket_id` already is the index) — `BASE_UNIT_SIZE` is 32 (one slab's word count, from `ConcurrentMapT`, §4.4), so `bucket_id * BASE_UNIT_SIZE + laneId` is exactly “the word this lane owns, in this bucket's base slab.”

- **Private allocation helpers**: two overloads of `allocateSlab` (one uses `global_allocator_ctx_` directly, one takes an external `AllocatorContextT&` — the latter is the one actually used everywhere, per §5.10's “always use a fresh local copy” pattern) and `freeSlab` (§5.8/§3.6).

8.5 Why GPU kernels use the context (not `GpuSlabHash` directly)

Purely a consequence of §8.1/§8.2: `GpuSlabHash` is a host object whose constructor and destructor perform `cudaMalloc/cudaFree` and whose data member `dynamic_allocator_` is a pointer into another host object's address space — none of that is meaningful or even well-defined inside `__device__` code. `GpuSlabHashContext` is what remains once all of that host-only machinery is stripped away: five small, trivially-copyable fields that fully describe “how to compute a bucket and fetch/modify a slab,” which is all a kernel actually needs.

8.6 What memory it owns vs. references

As established in §3.3: **nothing is owned**. `d_table_` is a copy of the pointer `GpuSlabHash` obtained from `cudaMalloc` (the context never calls `cudaMalloc/cudaFree` itself); `global_allocator_ctx_` is a value copy of `AllocatorContextT`, which itself only holds a non-owning pointer (`d_super_blocks_`) into memory that `SlabAllocLight` allocated (§5.1). If a `GpuSlabHashContext` instance outlives the `GpuSlabHash/SlabAllocLight` objects that set it up, its pointers become dangling — the class provides no protection against this, consistent with it being a lightweight, by-value-copied view rather than a resource owner.

8.7 How it is constructed and passed to kernels

Exactly one canonical instance is constructed per `GpuSlabHash`, as a plain member variable (`gpu_context_`, §8.3's table). It is populated once, via `initParameters`, at the end of `GpuSlabHash`'s constructor (§6.1), and never mutated again by `GpuSlabHash` itself. Every subsequent kernel launch simply passes this same member **by value** as the last kernel argument, e.g.:

```
// src/concurrent_map/cmap_implementation.cuh
search_table<KeyT, ValueT>
    <<<num_blocks, BLOCKSIZE_>>>(d_query, d_result, num_queries, gpu_context_);
```

CUDA's kernel-launch machinery copies `gpu_context_`'s bytes into the constant/parameter memory space visible to the launched grid; the `slab_hash` parameter inside `search_table` (§3.4) is therefore a fresh, independent copy for that launch — safe to read from many threads simultaneously, and cheap because the struct is small (a handful of `uint32_t`s, one pointer, and the allocator's own small context).

9. The SlabHash Variants

`SlabHashTypeT` (§4.4) names three variants. This section traces the source to establish, precisely, which of the three actually exist as working code.

9.1 `ConcurrentMap` — fully implemented (covered in §6 and §7)

The default, most complete variant: 15 key/value pairs per slab (`concurrent_slab`), supports `insertPair`, `insertPairUnique`, `searchKey`, `searchKeyBulk`, `countKey`, `deleteKey`, and a mixed-workload `batchedOperation`. This is the variant used by every gtest test in `test/cmap_test.cu` and by the benchmark harness.

9.2 `ConcurrentSet` — implemented, but with real gaps

`ConcurrentSet` stores only keys, using `key_only_slab<KeyT>` (§4.3): 31 keys + 1 next-pointer per 128-byte slab. Its traits class:

```
// src/slab_hash_global.cuh
template <typename KeyT>
class ConcurrentSetT {
public:
    static constexpr uint32_t A_INDEX_POINTER = 0xFFFFFFFF;
    static constexpr uint32_t EMPTY_INDEX_POINTER = 0xFFFFFFFF;
    static constexpr uint32_t BASE_UNIT_SIZE = 32;
    static constexpr uint32_t REGULAR_NODE_ADDRESS_MASK = 0x30000000;
    static constexpr uint32_t REGULAR_NODE_DATA_MASK = 0x7FFFFFFF;
    static constexpr uint32_t REGULAR_NODE_KEY_MASK = 0x7FFFFFFF;
    static constexpr uint32_t NEXT_PTR_LANE = 31;

    using SlabTypeT = key_only_slab<KeyT>;

    static std::string getName() { return std::string("ConcurrentSet"); }
};
```

What is shared with `ConcurrentMap`: the overall ballot-loop structure (§7.2) is structurally identical — `work_queue/last_work_queue/next`, the same `A_INDEX_POINTER/EMPTY_INDEX_POINTER` sentinels, the same `getPointerFromBucket/getPointerFromSlab` split, the same allocator calls (`allocateSlab`, `freeSlab`), and — unlike `ConcurrentMap` — `ConcurrentSet` actually calls the shared helper functions in `src/slab_hash_helper_methods.cuh` (`SlabHash_NS::findKeyOrEmptyPerWarp`, `findKeyPerWarp`) to do its per-slab ballot/mask logic, e.g.:

```
// src/concurrent_set/cset_warp_operations.cuh (inside insertKey)
int32_t dest_lane = SlabHash_NS::findKeyOrEmptyPerWarp<KeyT, ConcurrentSetT<KeyT>>(
    src_key, src_unit_data);
```

What is specialized: because there are no values, `ConcurrentSet`’s insert only ever CASes a single 32-bit key word (not a 64-bit key+value pair) —

```
// src/concurrent_set/cset_warp_operations.cuh (inside insertKey)
old_key = atomicCAS((unsigned int*)p,
    EMPTY_KEY,
    *reinterpret_cast<const uint32_t*>(
        reinterpret_cast<const unsigned char*>(&myKey)));
new_insertion = (old_key == EMPTY_KEY);
if (new_insertion || (old_key == src_key)) {
    to_be_inserted = false; // successful insertion
}
```

and note this logic is closer to `ConcurrentMap`’s `insertPairUnique` than to plain `insertPair`: it checks `old_key == src_key` (i.e., “did I lose the race to someone inserting this exact key?”) and treats that as success too — **`ConcurrentSet` has only one insert function, and it behaves like a “unique insert”**; there is no `ConcurrentMap`-style “always append a duplicate” mode for sets, which makes sense semantically (a mathematical set cannot contain the same element twice).

Interesting inconsistency found by reading the code closely: `GpuSlabHash<KeyT, ValueT, SlabHashTypeT::ConcurrentSet>`’s constructor sizes its `cudaMalloc` using the `ConcurrentMap`’s context type by mistake (or at least, this is what the source literally does):

```
// src/concurrent_set/cset_class.cuh, inside GpuSlabHash<..., ConcurrentSet>'s constructor
slab_unit_size_ =
    GpuSlabHashContext<KeyT, ValueT, SlabHashTypeT::ConcurrentMap>::getSlabUnitSize();
```

This does not currently cause a bug, because `getSlabUnitSize()` just returns `sizeof(SlabTypeT)`, and both `concurrent_slab` and `key_only_slab` happen to be exactly 128 bytes (§4.2, §4.3) — but it is worth flagging as a real, verifiable inconsistency in the source rather than something this document invented, since a reader who later changes one slab layout’s size without noticing this line would introduce a silent under-allocation.

Real functionality gaps, verified by reading the actual bodies (not assumed from the declarations in `cset_class.cuh`):

```
// src/concurrent_set/cset_class.cuh — GpuSlabHash<..., ConcurrentSet>
double computeLoadFactor(int flag) {}
void searchBulk(KeyT* d_query, ValueT* d_result, uint32_t num_queries) {}
void deleteIndividual(KeyT* d_key, uint32_t num_keys) {}
```

```
// src/concurrent_set/cset_warp_operations.cuh
template <typename KeyT, typename ValueT>
__device__ __forceinline__ bool
GpuSlabHashContext<KeyT, ValueT, SlabHashTypeT::ConcurrentSet>::searchKeyBulk(
    const uint32_t& laneId,
    const KeyT& myKey,
    const uint32_t bucket_id) {}
```

All four of these are declared with non-`void` return types (`computeLoadFactor` returns `double`, `searchKeyBulk` returns `bool`) but have **completely empty bodies** — they compile (falling off the end of a non-void function without a `return` is undefined behavior in C++, but `nvcc` allows it), but calling any of them would read an uninitialized return value.

ConcurrentSet therefore has no working `deleteIndividual`, no working bulk search, and no working load-factor calculation, unlike `ConcurrentMap`, which implements all three. This is stated here because the class *declares* these methods (in `cset_class.cuh`) in a way that looks, at a glance, identical to `ConcurrentMap`’s working versions — the gap is only visible by reading the actual `.cuh` implementation files, exactly the kind of thing this document is meant to surface.

When `ConcurrentSet` would be used: whenever the application only needs membership testing (does this key exist?), not an associated value — `test/concurrent_set.cu` is the repository’s own example, building a set of random keys and then searching for both present and absent keys.

9.3 PhaseConcurrentMap — declared as a type tag, but not implemented

As established in §4.3/§4.4: `SlabHashTypeT::PhaseConcurrentMap` exists as an enumerator, and `PhaseConcurrentMapT` exists as a traits class with its own `getTypeName()`:

```
// src/slab_hash_global.cuh
class PhaseConcurrentMapT {
public:
    static std::string getTypeName() { return std::string("PhaseConcurrentMap"); }
};
```

But this document’s full-repository search found:

- **No `GpuSlabHash<KeyT, ValueT, SlabHashTypeT::PhaseConcurrentMap>` specialization** anywhere — only the generic, undefined forward declaration `template <typename KeyT, typename ValueT, SlabHashTypeT SlabHashT> class GpuSlabHash;` exists for this tag; attempting to instantiate `GpuSlabHash<..., SlabHashTypeT::PhaseConcurrentMap>` would fail to compile (no member functions are defined for that specialization).
- **No `GpuSlabHashContext` specialization** for it either.
- **No kernel, warp-operation file, or `device//warp/` directory** for it (contrast with `concurrent_map/device`, `concurrent_map/warp`, `concurrent_set/cset_warp_operations.cuh`).
- The one struct that *looks* purpose-built for it, `phase_concurrent_slab<KeyT, ValueT>` (§4.3, separate keys+next-pointer region and values region), is likewise never referenced outside its own definition.

Conclusion, stated plainly rather than guessed at: `PhaseConcurrentMap` is a **placeholder for a variant that is not implemented in this snapshot of the repository**. The traits class and slab struct appear to be scaffolding left in place (perhaps for a “phase-concurrent” design where insertions and searches run in separate synchronized phases, given the name — but the source contains no comment confirming this intent, so this document does not assert it as fact). A reader should not expect to find working `PhaseConcurrentMap` insert/search/delete code anywhere in this repository.

9.4 Variant comparison summary

	ConcurrentMap	ConcurrentSet	PhaseConcurrentMap
Slab struct	<code>concurrent_slab</code> (15 kv-pairs + 2-word tail)	<code>key_only_slab</code> (31 keys + 1-word tail)	<code>phase_concurrent_slab</code> (defined, unused)
<code>insert</code>	<code>insertPair</code> (dups OK) + <code>insertPairUnique</code>	<code>insertKey</code> (unique only)	— (no implementation)
<code>search</code>	<code>searchKey</code> , working <code>searchKeyBulk</code>	<code>searchKey</code> works; <code>searchKeyBulk</code> empty body	—
<code>delete</code>	<code>deleteKey</code> , works	declared <code>deleteIndividual</code> , empty body	—
<code>count</code>	<code>countKey</code> , works	not present	—
Shares <code>slab_hash_helper_methods.cuh</code> ballot helpers	No (inlines its own logic)	Yes	—
Used by tests	<code>test/cmap_test.cu</code> , <code>test_slab_hash.cu</code> , <code>concurrent_map.cu</code> , <code>iterator_test.cu</code>	<code>test/concurrent_set.cu</code>	none
Used by <code>bench/</code>	Yes	No (bench only instantiates <code>ConcurrentMap</code>)	No

10. Tracing an Actual CUDA Kernel, Line by Line

This section picks `search_table` (`src/concurrent_map/device/search_kernel.cuh`) — the individual (non-bulk) search kernel — since it is short enough to cover completely while still touching the allocator-free, context-driven, warp-cooperative path.

10.1 The kernel itself

```
// src/concurrent_map/device/search_kernel.cuh
template <typename KeyT, typename ValueT>
__global__ void search_table(
    KeyT* d_queries,
    ValueT* d_results,
    uint32_t num_queries,
    GpuSlabHashContext<KeyT, ValueT, SlabHashTypeT::ConcurrentMap> slab_hash) {
    uint32_t tid = threadIdx.x + blockIdx.x * blockDim.x;
    uint32_t laneId = threadIdx.x & 0x1F;

    if ((tid - laneId) >= num_queries) {
        return;
    }

    KeyT myQuery = 0;
    ValueT myResult = static_cast<ValueT>(SEARCH_NOT_FOUND);
    uint32_t myBucket = 0;
    bool to_search = false;
    if (tid < num_queries) {
        myQuery = d_queries[tid];
        myBucket = slab_hash.computeBucket(myQuery);
        to_search = true;
    }

    slab_hash.searchKey(to_search, laneId, myQuery, myResult, myBucket);

    // writing back the results:
    if (tid < num_queries) {
        d_results[tid] = myResult;
    }
}
```

Line by line:

- `d_queries`, `d_results`, `num_queries`: flat device arrays/scalar, set up by `GpuSlabHash::searchIndividual` (§6.6) — `d_queries` was `cudaMemcpy`'d from the host's query array, `d_results` was allocated (and, in `gpu_hash_table::hash_search`, `cudaMemset` to `0xFF`) before the launch.
- `slab_hash`: the `GpuSlabHashContext` passed by value (§8.7) — this is where `computeBucket` and `searchKey` come from; note there is **no** `AllocatorContextT local_allocator_ctx` anywhere in this kernel, because search never allocates (§5.10).
- `tid = threadIdx.x + blockIdx.x * blockDim.x`: global thread index (§7.1).
- `laneId = threadIdx.x & 0x1F`: this thread's position (0–31) within its warp (§7.1).
- `if ((tid - laneId) >= num_queries) return;`: the whole-warp early exit (§7.1) — `tid - laneId` is this warp's lane-0 global index; if that's already past the end of the query array, this entire warp has nothing to do and can return immediately (no cooperative work is needed since no lane in the warp has valid work). Warps that are only *partially* out of range (some lanes valid, some not) do **not** take this early return — they fall through, and the invalid lanes get `to_search = false` below.
- `KeyT myQuery = 0; ValueT myResult = static_cast<ValueT>(SEARCH_NOT_FOUND); uint32_t myBucket = 0; bool to_search = false;`: default values for lanes that turn out not to have a valid query (so their participation in the warp-cooperative `searchKey` call below is well-defined but inert).
- `if (tid < num_queries) { myQuery = ...; myBucket = ...; to_search = true; }`: only threads with an actual valid, in-range query load `d_queries[tid]`, call `slab_hash.computeBucket(myQuery)` (§6.2 — the $((\text{hash}_x \wedge \text{key}) + \text{hash}_y) \% \text{PRIME_DIVISOR}) \% \text{num_buckets}$ formula) to get their bucket index, and set `to_search = true` so the warp-cooperative loop below knows this lane needs work.
- `slab_hash.searchKey(to_search, laneId, myQuery, myResult, myBucket);`: the call into `GpuSlabHashContext::searchKey` (§10.2) — every one of the 32 threads in the warp calls this **together**, passing their own `to_search`/`laneId`/`myQuery`/`myResult`/`myBucket`. `to_search` and `myResult` are passed by reference (`bool&`, `ValueT&`) — `searchKey` mutates them in place: it flips `to_search` to `false` once a lane's query has been resolved (found or confirmed absent), and writes the found value (or leaves the `SEARCH_NOT_FOUND` default) into `myResult`.
- `if (tid < num_queries) { d_results[tid] = myResult; }`: only valid threads write their (now-resolved) result back to global memory.

10.2 Following the call into `GpuSlabHashContext::searchKey`

```

// src/concurrent_map/warp/search.cuh
template <typename KeyT, typename ValueT>
__device__ __forceinline__ void
GpuSlabHashContext<KeyT, ValueT, SlabHashTypeT::ConcurrentMap>::searchKey(
    bool& to_be_searched,
    const uint32_t& laneId,
    const KeyT& myKey,
    ValueT& myValue,
    const uint32_t bucket_id) {
    using SlabHashT = ConcurrentMapT<KeyT, ValueT>;
    uint32_t work_queue = 0;
    uint32_t last_work_queue = work_queue;
    uint32_t next = SlabHashT::A_INDEX_POINTER;

    while ((work_queue = __ballot_sync(0xFFFFFFFF, to_be_searched))) {
        next = (last_work_queue != work_queue) ? SlabHashT::A_INDEX_POINTER : next;
        uint32_t src_lane = __ffs(work_queue) - 1;
        uint32_t src_bucket = __shfl_sync(0xFFFFFFFF, bucket_id, src_lane, 32);
        uint32_t wanted_key = __shfl_sync(0xFFFFFFFF,
            *reinterpret_cast<const uint32_t*>(
                reinterpret_cast<const unsigned char*>(&myKey)),
            src_lane, 32);
        const uint32_t src_unit_data = (next == SlabHashT::A_INDEX_POINTER)
            ? *(getPointerFromBucket(src_bucket, laneId))
            : *(getPointerFromSlab(next, laneId));
        int found_lane = __ffs(__ballot_sync(0xFFFFFFFF, src_unit_data == wanted_key) &
            SlabHashT::REGULAR_NODE_KEY_MASK) - 1;
        if (found_lane < 0) {
            uint32_t next_ptr = __shfl_sync(0xFFFFFFFF, src_unit_data, 31, 32);
            if (next_ptr == SlabHashT::EMPTY_INDEX_POINTER) {
                if (laneId == src_lane) {
                    myValue = static_cast<ValueT>(SEARCH_NOT_FOUND);
                    to_be_searched = false;
                }
            } else {
                next = next_ptr;
            }
        } else {
            uint32_t found_value = __shfl_sync(0xFFFFFFFF, src_unit_data, found_lane + 1, 32);
            if (laneId == src_lane) {
                myValue = *reinterpret_cast<const ValueT*>(
                    reinterpret_cast<const unsigned char*>(&found_value));
                to_be_searched = false;
            }
        }
        last_work_queue = work_queue;
    }
}

```

Every construct here was introduced generically in §7 — this is the same code already quoted there, now traced in the specific context of “how a single `search_table` kernel launch resolves one query.” Two lines deserve a closer look:

- `*reinterpret_cast<const uint32_t*>(reinterpret_cast<const unsigned char*>(&myKey))`: this reinterprets `myKey` (whatever `KeyT` is — the benchmark and tests use `uint32_t`) as a raw `uint32_t` bit pattern, so that it can be shuffled with `__shfl_sync`, which only operates on 32-bit register values, and so that the comparison `src_unit_data == wanted_key` is a plain integer comparison against the raw word read from the slab (which is itself stored as a raw `uint32_t`, per §4.2’s layout).
- `__shfl_sync(0xFFFFFFFF, src_unit_data, found_lane + 1, 32)`: once a key match is found at lane `found_lane`, the corresponding `value` is one lane over — a direct consequence of the `k0, v0, k1, v1, ...` interleaved layout (§4.2) — so shuffling from `found_lane + 1` reads the value word from whichever lane currently holds it, and delivers it to every lane (though only `laneId == src_lane` actually keeps it).

10.3 The full chain, in one diagram

```

Application (test/bench code)
| calls gpu_hash_table::hash_search(h_query, h_result, num_queries)
|
v
gpu_hash_table::hash_search [src/gpu_hash_table.cuh]
| cudaMemcpy(d_query, h_query, ..., H2D); cudaMemcpy(d_result, 0xFF, ...)
| slab_hash->searchIndividual(d_query, d_result, num_queries)
|
v
GpuSlabHash::searchIndividual [src/concurrent_map/cmap_implementation.cuh]
| num_blocks = ceil(num_queries / 128)
| search_table<<<num_blocks, 128>>>(d_query, d_result, num_queries, gpu_context_)
|
v
search_table (__global__ kernel, runs on GPU) [src/concurrent_map/device/search_kernel.cuh]
| tid, laneId computed; early-exit if whole warp is out of range
| myQuery, myBucket = slab_hash.computeBucket(myQuery); to_search = true
| slab_hash.searchKey(to_search, laneId, myQuery, myResult, myBucket)
|
v
GpuSlabHashContext::searchKey (__device__, warp-cooperative) [src/concurrent_map/warp/search.cuh]
| ballot for work_queue; pick src_lane; shuffle its bucket+key to everyone
| every lane reads word `laneId` of the current slab:
|   first iteration -> getPointerFromBucket(src_bucket, laneId)
|
v
GpuSlabHashContext::getPointerFromBucket [src/concurrent_map/cmap_class.cuh]
| reinterpret_cast<uint32_t*>(d_table_) + bucket_id * BASE_UNIT_SIZE + laneId
|
v
GPU global memory: d_table_[bucket_id], word `laneId` (the actual key/value/next data)
|
| ...ballot on src_unit_data == wanted key, masked to key lanes...
| if found: shuffle value from found_lane+1, deliver to src_lane
| if not found and lane-31's word != EMPTY_INDEX_POINTER:
|   next = that word; loop continues, next iteration reads
|   getPointerFromSlab(next, laneId) instead of getPointerFromBucket
|
v
GpuSlabHashContext::getPointerFromSlab -> AllocatorContextT::getPointerFromSlab -> addressDecoder
| [src/concurrent_map/cmap_class.cuh -> SlabAlloc/src/slab_alloc.cuh, §5.4]
|
v
GPU global memory: d_super_blocks_[decoded offset + laneId] (a chained/overflow slab's word)
|
|
v
Result: myValue set (found) or SEARCH_NOT_FOUND (chain exhausted) -> to_be_searched = false
| back up through searchKey -> search_table writes d_results[tid] = myResult
|
v
GpuSlabHash::searchIndividual returns -> gpu_hash_table::hash_search does cudaMemcpy D2H

```

11. Memory Layout

This section consolidates the memory-layout facts established piecemeal in §3–§5 into one place, with the actual types and fields named throughout.

11.1 The two GPU allocations, side by side

ALLOCATION #1: `GpuSlabHash::d_table_`
`cudaMalloc(&d_table_, slab_unit_size_ * num_buckets_)`
where `slab_unit_size_ = sizeof(concurrent_slab<K,V>)`
= 128 bytes

bucket 0	bucket 1	bucket 2	...	bucketN-1
128 bytes	128 bytes	128 bytes		128 bytes

one base slab per bucket, contiguous, index = bucket_id
(`GpuSlabHashContext::getPointerFromBucket` does simple pointer arithmetic: `bucket_id * 32 + laneId`, no allocator involved)

ALLOCATION #2: `SlabAllocLight::d_super_blocks_`
`cudaMalloc(&d_super_blocks_, SUPER_BLOCK_SIZE_ * num_super_blocks_ * 4 bytes)`

super-block 0	super-block 1	...	SB 31
(each: bitmaps region, then 256 memory blocks x 1024 slabs of 128 bytes each)			

ALL overflow/chained slabs for EVERY bucket of EVERY `GpuSlabHash` sharing this allocator live somewhere in here (`GpuSlabHashContext::getPointerFromSlab` decodes a 32-bit address into an offset here, via `addressDecoder`, §5.4)

11.2 Bucket storage

A bucket has no separate “header” object — it is index `bucket_id` into `d_table_`, and `computeBucket` (§6.2) is the only function that produces a `bucket_id`. `getPointerFromBucket(bucket_id, laneId)` (§8.4) computes:

```
address = (uint32_t*)d_table_ + bucket_id * BASE_UNIT_SIZE + laneId
         = (uint32_t*)d_table_ + bucket_id * 32             + laneId
```

— i.e. bucket `bucket_id`’s base slab starts at word offset `bucket_id * 32` (byte offset `bucket_id * 128`) inside `d_table_`, and lane `laneId` owns word `laneId` within that 32-word slab.

11.3 Slab storage (chained/overflow slabs)

Covered in full in §5.2–§5.4. In one line: a chained slab’s `SlabAddressT` encodes (`super_block_index`, `mem_block_index`, `mem_unit_index`), and `addressDecoder` + `getPointerFromSlab` turn that into `d_super_blocks_ + (super_block_index * SUPER_BLOCK_SIZE_) + (mem_block_index's storage offset, which already skips past all 256 bitmaps in that super-block) + (mem_unit_index * 32) + laneId`.

11.4 Key/value storage within one slab

Established byte-for-byte in §4.2: `concurrent_slab` interleaves `k0,v0,k1,v1,...,k14,v14` across lanes 0–29, with lane 30 unused by current code and lane 31 holding the next-pointer. `ConcurrentSet`’s `key_only_slab` instead uses lanes 0–30 for 31 keys and lane 31 for the next-pointer (§4.3, §9.2).

11.5 Pointers / indices

There are exactly two kinds of “pointer” in this codebase, and they are not interchangeable:

- A **`SlabAddressT` / `SlabAllocAddressT`** (`uint32_t`): an *opaque, encoded* address into the allocator’s `d_super_blocks_`, produced by `warpAllocate` (§5.3/§5.5) and consumed only by `addressDecoder` / `getPointerFromSlab`. This is what’s stored in a slab’s lane-31 “next” word.
- A **raw `uint32_t*`**: an actual, dereferenceable GPU pointer, returned by `getPointerFromBucket` or `getPointerFromSlab` — this is what insert/ search/delete/count actually read from or write to with `atomicCAS` / `atomicExch`/plain loads.

The two sentinel values that appear in “next” words, `A_INDEX_POINTER (0xFFFFFFFF)` and `EMPTY_INDEX_POINTER (0xFFFFFFFF)`, are *not* `SlabAddressT`s that get decoded — they are special-cased directly in every ballot loop (`next == SlabHashT::A_INDEX_POINTER` means “read the bucket’s base slab, not a chained one”; `next_ptr == SlabHashT::EMPTY_INDEX_POINTER` means “chain ends here”) before `getPointerFromSlab` would ever be called on them.

11.6 Allocator metadata

The only allocator metadata that lives in GPU memory (as opposed to a warp’s registers, i.e. the `SlabAllocLightContext` fields in §4.5) is the **per-memory-block bitmap** — 32 words at the start of each memory block’s region within a super-block (§5.2’s diagram), one bit per memory unit, tested/set via `atomicCAS` / `atomicAnd` (§5.5, §5.8). There is no separate

free-list, no reference counts, and no metadata stored per-slab beyond that single bitmap bit (a slab does not “know” whether it’s allocated — the allocator’s bitmap is the only source of truth for that).

11.7 GPU memory ownership recap

Restating §3.3/§8.6 in memory-layout terms: `d_table_` is owned by `GpuSlabHash` (freed in `~GpuSlabHash()` via `cudaFree(d_table_)`); `d_super_blocks_` is owned by `SlabAllocLight` (freed in its own destructor, covered structurally in §5.1 — the allocator’s `.cuh` mirrors `GpuSlabHash`’s constructor/destructor pattern for its one `cudaMalloc/cudaFree` pair). Both `GpuSlabHashContext` and `SlabAllocLightContext` hold only **copies of the pointers**, never allocate or free anything themselves (§8.6).

12. Concurrency

Every use of an atomic or warp-collective operation in this codebase exists to prevent a specific, nameable race. This section goes through each one and explains the race, not just the instruction.

12.1 Linking a new slab onto a chain: `atomicCAS` on the next-pointer word

```
// src/concurrent_map/warp/insert.cuh (inside insertPair, when no slot was
// found and next == EMPTY_INDEX_POINTER)
uint32_t new_node_ptr = allocateSlab(local_allocator_ctx, laneId);
if (laneId == 31) {
    uint32_t temp = atomicCAS((unsigned int*)p, SlabHashT::EMPTY_INDEX_POINTER, new_node_ptr);
    if (temp != SlabHashT::EMPTY_INDEX_POINTER) {
        freeSlab(new_node_ptr);
    }
}
```

The race without the CAS: two different warps could simultaneously discover that the same bucket's chain currently ends in `EMPTY_INDEX_POINTER` (e.g., two warps both inserting into a bucket whose chain is currently full but has no next slab yet). If both warps simply wrote their own newly allocated slab's address into the tail's next-pointer word with a plain store, whichever write happened to land physically last would silently overwrite the other — the loser's new slab would be allocated, but never linked into the chain, becoming permanently unreachable *and* still marked as "in use" in the allocator's bitmap (a leak). `atomicCAS` prevents this by making the write conditional: it only succeeds if the word still contains `EMPTY_INDEX_POINTER` (the value the warp last observed) — the second warp's CAS necessarily fails (`temp != EMPTY_INDEX_POINTER`, because the winner's value is there now), and the losing warp explicitly frees its now-orphaned slab via `freeSlab` (§5.8) instead of leaking it, and its `next` variable picks up the winner's slab on the *next* loop iteration instead (implicit, since the loop reruns and re-reads the now-updated pointer).

12.2 Inserting a key/value pair: 64-bit `atomicCAS`

```
// src/concurrent_map/warp/insert.cuh (inside insertPair)
old_key_value_pair = atomicCAS((unsigned long long int*)p, EMPTY_PAIR_64, /* packed key|value */);
if (old_key_value_pair == EMPTY_PAIR_64)
    to_be_inserted = false;
```

The race without the CAS: two different warps' insert operations could both have found the *same* empty slot inside the *same* slab (this can genuinely happen: the ballot in §6.4/§7.3 that identifies an empty lane is only a *snapshot*; two different warps process this slab in overlapping time windows and could both compute the same `dest_lane` before either has written anything). If both then did a plain, unconditional write of their own key/value pair into that slot, one insertion would silently vanish — whichever write landed last would win, and the other lane's `to_be_inserted` flag would incorrectly become `false` (or worse, stay `true` and loop looking for a *different* slot, while its original key was actually overwritten by the other warp and lost). The 64-bit CAS turns this into a detectable, resolvable race: only the warp whose CAS actually observes `EMPTY_PAIR_64` (i.e., wins the race) succeeds and clears its `to_be_inserted` flag; the loser's CAS returns whatever the winner just wrote (not `EMPTY_PAIR_64`), so the loser correctly concludes this slot is now taken and its own key still needs a home — the ballot loop simply continues, discovering the slot is no longer empty on the next fetch. Additionally, doing key+value as **one** 64-bit operation (rather than two separate 32-bit writes) prevents a second, different race: without it, a concurrent reader could observe a freshly written key with the *old* (still-empty, or some other stale) value, or vice versa — a torn read across the two fields. The single 64-bit CAS makes the pair's insertion atomic as a unit.

12.3 Deleting a key: 64-bit `atomicExch`

```
// src/concurrent_map/warp/delete.cuh
uint64_t old_pair = atomicExch((unsigned long long int*)p, EMPTY_PAIR_64);
successful_deletion = (static_cast<uint32_t>(old_pair) == src_key);
```

The race without the atomic exchange: if deletion were a plain, non-atomic write of `EMPTY_PAIR_64`, it could race against another warp's concurrent `insertPair` targeting the *same* slot (e.g., that slot was just freed by this delete and immediately reused by a racing insert) — a non-atomic write could interleave at the byte level with the insert's own 64-bit write, producing a corrupted, half-old/half-new key+value pair visible to any thread that reads it afterward. `atomicExch` guarantees the 64 bits are swapped as one indivisible unit, and — just as importantly — returns the *previous* contents atomically, which is how `deleteKey` verifies (after the fact) that it actually deleted the intended key rather than a different key that happened to occupy that slot lane at the moment of the exchange (e.g., if the warp's own ballot-based match was based on a now-stale read, because another warp deleted-and-reinserted a different key into that exact slot in between).

12.4 Allocating a slab: `atomicCAS` on a bitmap word

Already explained in §5.5 mechanically; restated here as a race: **the race without the CAS** is two different warps (both resident in the same memory block, both observing the same free bit in the same lane's bitmap word) both deciding to claim memory unit N — without a CAS, both would believe they successfully allocated the same 128-byte region, and would then both write live key/value data into what they each think is *their own, exclusively owned* slab, silently corrupting each other's entries. `atomicCAS(bitmap_word_ptr, resident_bitmap_, resident_bitmap_ | (1 << empty_lane))` only succeeds for the warp whose *locally cached* copy of the bitmap word (`resident_bitmap_`) still matches what's actually in memory — the loser's CAS fails, it refreshes its local cache from the (now-updated) return value, and the `while` loop in `warpAllocate` simply tries again, likely finding a *different* empty bit this time.

12.5 Warp-collective ballots/shuffles themselves are a synchronization mechanism

`__ballot_sync / __shfl_sync` are not just “communication” primitives — their `_sync` suffix means they include an implicit **warp-level barrier**: every lane named in the mask argument (`0xFFFFFFFF` = all 32, throughout this codebase) must reach that instruction before any of them proceeds past it. This is what guarantees, for example, that `work_queue` (§7.2) reflects every lane's *current to_be_searched / to_be_inserted* value at the same logical instant — without this synchronization, a warp could compute a `work_queue` from a mix of lanes at different loop iterations (some lanes having already updated their flag for this round, others not yet), silently corrupting the whole warp's shared understanding of “who still has work” and “which slab are we currently looking at.”

12.6 Memory ordering

This document did not find any explicit memory-fence instructions (`__threadfence()`, `__threadfence_block()`, etc.) anywhere in either `SlabHash/src` or `SlabAlloc/src`. The correctness of this design instead relies on: (a) `atomicCAS / atomicExch` on NVIDIA GPUs being globally visible, sequentially consistent *with respect to that specific memory location* the moment they complete (the CUDA memory model guarantees this for atomics without requiring a separate fence for the atomic's own visibility), and (b) every genuinely “was this write visible yet” question in the algorithm being resolved by re-reading the exact word that was atomically written (e.g., re-fetching a slab's next-pointer word, or a bitmap word, rather than assuming an ordering relationship with some other, non-atomically-updated piece of state). This document does not assert that this is provably sufficient for every possible interleaving on every GPU architecture — only that no additional fences are present in the source, and that the algorithm's design (single-word atomics as the sole cross-warp communication channel) is consistent with not needing them for the specific races enumerated above.

13. The Benchmark Code

13.1 How benchmarks are configured

`bench/main_benchmarks.cu`'s `main()` is a command-line-driven dispatcher. It reads a long list of flags via the small `CommandLine.h` helper (§2.3's sibling file, `src/CommandLine.h` — a simple `argv` scanner, `cmdOptionExists` / `getCmdOption`), then switches on a `-mode` integer into one of four experiment functions defined in `bench/experiments.cuh`:

```
// bench/main_benchmarks.cu
using KeyT = uint32_t;
using ValueT = uint32_t;

switch (mode) {
case 0: // singleton experiment
    singleton_experiment<KeyT, ValueT>(num_keys, num_queries, expected_chain,
                                       filename, device_idx, existing_ratio,
                                       num_iter, /*run_cudpp = */ false, verbose);
    break;
case 1: // bulk build, num elements fixed, load factor changing
    load_factor_bulk_experiment<KeyT, ValueT>(...);
    break;
case 2: // bulk build, load factor fixed, num elements changing
    build_search_bulk_experiment<KeyT, ValueT>(...);
    break;
case 3: // concurrent experiment:
    concurrent_batched_op_load_factor_experiment<KeyT, ValueT>(...);
    break;
default:
    std::cout << "Error: invalid mode." << std::endl;
    break;
}
```

Both `KeyT` and `ValueT` are hard-coded to `uint32_t` for every benchmark mode — the CLI doesn't let you benchmark a different key/value type. Notable flags: `-num_key`, `-num_query`, `-expected_chain` (target average chain length, used to size `num_buckets`, see §13.2), `-query_ratio` (fraction of queries that should actually be present in the table — `existing_ratio`), `-nStart` / `-nEnd` (powers of two controlling key-count sweeps for modes 2/3), `-num_batch` / `-init_batch` / `-insert_ratio` / `-delete_ratio` / `-search_exist_ratio` (mode 3's mixed-workload knobs), `-device`, `-iter` (repeat count for averaging), and `-filename` (JSON output path, defaulting to a timestamped file under `bench/`).

13.2 How the number of buckets is chosen: `expected_chain`

```
// bench/experiments.cuh (this exact pattern recurs in singleton_experiment,
// load_factor_bulk_experiment, and build_search_bulk_experiment)
const uint32_t num_elements_per_unit = 15;
uint32_t expected_elements_per_bucket = expected_chain * num_elements_per_unit;
uint32_t num_buckets = (num_keys + expected_elements_per_bucket - 1) /
    expected_elements_per_bucket;
```

`num_elements_per_unit = 15` is exactly `ConcurrentMapT::NUM_ELEMENTS_PER_SLAB` (§4.2) — hard-coded here rather than referenced symbolically, but numerically identical. This computes “how many buckets are needed so that, on average, each bucket's *base slab alone* holds `expected_chain` times its 15-slot capacity worth of keys” — directly connecting the benchmark's bucket-count choice back to the `concurrent_slab` layout from §4.2. An `expected_chain` of `1.0`, for example, aims for each bucket's base slab to be, on average, completely full with no overflow slabs needed at all; values above `1.0` deliberately force some fraction of buckets to grow chains via the allocator (§5), which is why this parameter directly controls how much allocator activity a given benchmark run exercises.

13.3 What operations are measured, and how

Every experiment function follows the same shape: construct a `gpu_hash_table<KeyT, ValueT, SlabHashTypeT::ConcurrentMap>` (the exact class from §3.1), generate random keys via `bench/batched_data_gen.h`'s `generate_uniform_unique_keys`, call the relevant `hash_*` method (`hash_build`, `hash_search`, etc. — the same public API traced in §6.5–§6.7), and record the `float` return value each of those methods produces — every `hash_*` method in `gpu_hash_table.cuh` times its own kernel launch with `std::chrono`/CUDA events and returns milliseconds directly, e.g.:

```
// src/gpu_hash_table.cuh (representative pattern used by hash_build, hash_search, etc.)
float hash_build(KeyT* h_key, ValueT* h_value, uint32_t num_keys) {
    float time_ms;
    CHECK_CUDA_ERROR(cudaSetDevice(device_idx));
    ...
    timer.startTimer();
    slab_hash->buildBulk(d_key_, d_value_, num_keys);
    time_ms = timer.stopTimer();
    return time_ms;
}
```

The experiment functions then convert this into a throughput figure — typically **million operations per second (MOPS)**, computed as `num_keys / (time_ms * 1000.0)` — and log it (either to `stdout` if `verbose`, or into a `rapidjson::Document` that gets written to the `-filename` JSON path).

13.4 What the different benchmark modes mean

- **Mode 0 — singleton_experiment**: the simplest case, a single build-then-search-then-(optionally)-delete run at one fixed key count and one fixed `expected_chain`, repeated `num_iter` times for averaging. This is the mode used for a quick sanity/performance check.
- **Mode 1 — load_factor_bulk_experiment**: fixed number of keys, but the table is built incrementally in `lf_bulk_num_sample` steps of `lf_bulk_step` fraction each, measuring build+search throughput *as a function of load factor* — i.e. this produces the classic “throughput vs. how full the table is” curve.
- **Mode 2 — build_search_bulk_experiment**: fixed `expected_chain` (so load factor is held roughly constant), but the *number of keys* is swept across powers of two from `1 << n_start` to `1 << n_end` — this produces a “throughput vs. scale” curve.
- **Mode 3 — concurrent_batched_op_load_factor_experiment**: exercises `gpu_hash_table`’s `hash_batched_operation` / `GpuSlabHash::batchedOperation` path (§6, the `batchedOperation` declared in `cmap_class.cuh` and implemented via `concurrent_kernel.cuh`’s `batched_operations` kernel) — a single kernel launch that mixes insert/search/delete operations together in one batch, according to `insert_ratio/delete_ratio/search_exist_ratio`, run over `num_batch` batches. This is the mode that actually stresses the concurrency mechanisms of §12 the hardest, since inserts, deletes, and searches can all be racing against each other within the same kernel launch, on overlapping buckets.

13.5 Which functions actually execute the hash-table operations

Every experiment function is a **thin driver** — it never touches `GpuSlabHash`, `GpuSlabHashContext`, or any kernel directly. It always goes through `gpu_hash_table`’s public methods (`hash_build`, `hash_build_with_unique_keys`, `hash_search`, `hash_search_bulk`, `hash_delete`, `hash_batched_operation`, `hash_count`, defined in `src/gpu_hash_table.cuh`), which is the same application-level wrapper class discussed in §3.1 — reinforcing that the benchmark harness is a *consumer* of the public API, not a special/privileged caller.

13.6 `bencher.py`

`bench/bencher.py` is a small Python script that shells out to the compiled `build/bin/benchmark` executable (via `subprocess`), passing different `-mode`/parameter combinations, and can optionally parse the resulting JSON output for downstream plotting. It is a convenience wrapper for running sweeps of the C++ benchmark binary — it contains no hash-table logic of its own.

14. The Tests

14.1 test/cmap_test.cu — the gtest suite (the primary correctness tests)

Six `TEST(...)` cases, each following the same pattern: build a `gpu_hash_table<uint32_t, uint32_t, SlabHashTypeT::ConcurrentMap>`, feed it known key/value data, run one operation, and cross-check the GPU result against a **host-side** `std::unordered_map` built from the same data — the `std::unordered_map` acts as the ground truth (“oracle”) the GPU implementation is checked against.

- **TEST(ConcurrentMap, Construction)**: the minimal smoke test — builds a 100-bucket table with a 10-key/value seed set, just to confirm construction and `hash_build` don’t crash. Exercises: `GpuSlabHash`’s constructor (§6.1) and `insertPair` (§6.5) via `buildBulk`.
- **TEST(BulkBuild, IndividualSearch)**: builds 137 sequential keys (13, 14, ..., 149) into a table with only **2 buckets** — deliberately small, forcing long chains and exercising the allocator (§5) heavily — then shuffles the same keys as queries and checks every result against an `unordered_map`. Exercises: `insertPair`, `searchKey` (the individual, non-bulk path, §10) via `hash_search`.
- **TEST(BulkBuild, BulkSearch)**: identical setup, but calls `hash_search_bulk` instead — exercising `searchKeyBulk` (§6.6’s `#pragma unroll`-based variant) instead of `searchKey`.
- **TEST(BulkBuild, IndividualCount)**: generates a random *count* for each of 2,014 unique keys (0 to 31 copies each, via `rng() % max_count`), inserts all the resulting duplicate key/value pairs with plain `insertPair` (via `hash_build`, which allows duplicates, §6.5), then queries each unique key’s count via `hash_count` and checks it against the count that was actually generated for it. Exercises: `countKey` (§6.8), and specifically the fact that `insertPair` allows the same key to appear in a chain multiple times.
- **TEST(UniqueBulkBuild, IndividualCount)**: the same random-count setup, but built with `hash_build_with_unique_keys` (→ `insertPairUnique`, §6.5) instead of plain `hash_build` — since only the *first* occurrence of each key is actually stored, the expected count for every key becomes either 0 or 1 (`(count_map[h_query[i]] != 0) ? 1 : 0`), directly testing the difference between `insertPair` and `insertPairUnique`’s deduplication behavior.
- **TEST(BulkBuild, IndividualDelete)**: builds 137 keys, then deletes a larger, overlapping/shuffled set of “candidate” keys (4x the original count, so many candidates were never actually inserted) via `hash_delete` (→ `deleteKey`, §6.7), then re-searches all the *original* keys and checks that deleted ones are gone (matching an `unordered_map` with the deleted keys `.erase()` d) while non-deleted ones are untouched. This specifically exercises `deleteKey`’s behavior when asked to delete a key that was never present (it should simply not find it and move on, without corrupting the table).

14.2 The standalone (non-gtest) test executables

- **test/test_slab_hash.cu**: the simplest possible end-to-end smoke test — no gtest, just a plain `main()` that constructs a `GpuSlabHash/DynamicAllocatorT` pair directly (not via `gpu_hash_table`), inserts a handful of keys, searches for them, and prints pass/fail to `stdout`. Useful as the very first thing to read when learning the raw (non-wrapped) API, since it skips `gpu_hash_table`’s extra bookkeeping (persistent device buffers, timers) and calls `GpuSlabHash::buildBulk/searchIndividual` almost directly.
- **test/concurrent_map.cu**: a similar standalone driver, but exercising `gpu_hash_table<KeyT, ValueT, SlabHashTypeT::ConcurrentMap>`’s public API end-to-end (build, search, delete) with manual result-checking printed to `stdout` rather than gtest assertions.
- **test/concurrent_set.cu**: the equivalent standalone driver for `SlabHashTypeT::ConcurrentSet` (§9.2) — instantiates `gpu_hash_table<KeyT, KeyT, SlabHashTypeT::ConcurrentSet>` (note `KeyT` used for *both* template arguments, per §9.2’s finding), builds a set of random keys, and searches for a mix of present and absent keys, printing whether each result matches expectations.
- **test/iterator_test.cu**: exercises `SlabIterator` — builds a `gpu_hash_table<KeyT, KeyT, SlabHashTypeT::ConcurrentSet>` with `identity_hash = true` (so `computeBucket` becomes a plain `key % num_buckets`, making bucket assignment fully predictable for the test), then launches a small custom kernel that constructs a `SlabIterator<KeyT>` directly from the table’s `GpuSlabHashContext` and repeatedly calls its `next()` method (`src/slab_iterator.cuh`) to walk every key in the table, printing each one found — a direct end-to-end check that the iterator correctly walks both base slabs and chained/overflow slabs across every bucket.

14.3 What running the tests actually exercises, end to end

Every test in `test/`, gtest-based or standalone, ultimately drives the same call chain traced in §6 and §10 — `gpu_hash_table` (or, for `test_slab_hash.cu`, `GpuSlabHash` directly) → kernel launch → warp-cooperative context methods → allocator (for anything that inserts). There is no separate “mock” or “test-only” code path anywhere in the repository — the tests exercise exactly the same production code the benchmarks and any downstream application would use.

15. The Build System

15.1 The root CMakeLists.txt

```
cmake_minimum_required (VERSION 3.8 FATAL_ERROR)
project (SlabHash)

find_package(CUDA 8.0 REQUIRED)

option(CMAKE_VERBOSE_MAKEFILE ON)
option(DGTEST, "DGTEST" ON)

set(CUDA_NVCC_FLAGS -std=c++11)
set (CMAKE_CXX_STANDARD 11)

set(CMAKE_BUILD_TYPE "Release")
set(CMAKE_RUNTIME_OUTPUT_DIRECTORY ${CMAKE_BINARY_DIR}/bin)

set(GENCODE_SM30 -gencode=arch=compute_30,code=sm_30 -gencode=arch=compute_30,code=compute_30)
set(GENCODE_SM35 -gencode=arch=compute_35,code=sm_35 -gencode=arch=compute_35,code=compute_35)
... (SM37, SM50, SM60, SM61, SM70, SM71, SM75 follow the same pattern)

option(SLABHASH_GENCODE_SM30 "GENCODE_SM30" OFF)
option(SLABHASH_GENCODE_SM35 "GENCODE_SM35" ON)
... (all other SMs default OFF)

if (SLABHASH_GENCODE_SM35)
    set(GENCODE ${GENCODE} ${GENCODE_SM35})
endif (SLABHASH_GENCODE_SM35)
... (one such if-block per architecture)

include_directories(SlabAlloc/src)
include_directories(src src/concurrent)
include_directories(ThirdParty/rapidjson/include)
include_directories(ThirdParty/googletest/googletest)
include_directories(ThirdParty/googletest/googletest/include)
add_subdirectory(ThirdParty/googletest/googletest)
add_subdirectory(test)
add_subdirectory(bench)

if (DGTEST)
    enable_testing()
endif()
```

- **CUDA configuration:** `find_package(CUDA 8.0 REQUIRED)` — the classic (pre-“first-class CUDA language”) CMake `FindCUDA` module, requiring at least CUDA 8.0; the CUDA C++ standard is fixed to C++11 (`set(CUDA_NVCC_FLAGS -std=c++11)`).
- **GPU architecture flags:** a manual list of `-gencode` flag sets, one per compute-capability generation from `sm_30` through `sm_75`, each guarded by its own `SLABHASH_GENCODE_SM*` CMake option — **only SM35 is ON by default**; every other architecture is `OFF`. This is exactly why the README’s compilation step 3 explicitly tells the user to turn on the option matching their GPU (e.g. `SLABHASH_GENCODE_SM35`), either by editing `CMakeLists.txt` directly or via `ccmake` ... Building for the wrong (or no) architecture would either fail to run on the actual GPU or simply not compile any device code for it.
- **Compiler flags:** `CMAKE_BUILD_TYPE` is hard-coded to `"Release"` (not configurable via a `-DCMAKE_BUILD_TYPE=...` override, since it’s `set()` unconditionally rather than as a cached option with a default) — optimized builds only, no separate debug configuration is wired up.
- **Include paths:** `SlabAlloc/src` (the allocator, §5.1), `src` (SlabHash proper) and, notably, `src/concurrent` — **this directory does not actually exist** in the repository (the real directories are `src/concurrent_map` and `src/concurrent_set`, §2.1); this appears to be a stale/incorrect include path left over from an earlier directory layout. It does not break the build, because every file that needs to reach into `concurrent_map/` or `concurrent_set/` does so via an explicit relative `#include "concurrent_map/cmap_class.cuh"` from `slab_hash.cuh`, which resolves correctly against the (real, present) `src` include directory alone.
- **add_subdirectory:** only `ThirdParty/googletest/googletest`, `test`, and `bench` are added as subdirectories. `add_subdirectory(SlabAlloc)` and `add_subdirectory(src)` are **conspicuously absent** — meaning `SlabAlloc/CMakeLists.txt` and `src/CMakeLists.txt` (§15.2) are **never invoked by the root build at all**. This is a real, verifiable property of the build graph, not an oversight in this document: it makes sense once you notice that `slab_hash.cuh` and everything it transitively includes is entirely **templates and `__forceinline` / inline functions in headers** — there is nothing to separately compile into a library; `test/` and `bench/`’s own `.cu` files simply `#include "slab_hash.cuh"` directly and get fully self-contained translation units, so no separate `slab_hash` library ever needs to be built or linked.

15.2 src/CMakeLists.txt — defined, but not actually built by the root project

```

set(CUHFILES
  slab_hash_global.cuh
  slab_hash.cuh)

cuda_add_library(slab_hash STATIC
  ${CUHFILES}
  ${CUHFILES}
  OPTIONS ${GENCODE} ${VERBOSE_PTXAS})

```

This file defines a `slab_hash` static library target — but as established in §15.1, the root `CMakeLists.txt` never calls `add_subdirectory(src)`, so this target is **never actually configured or built** when you run `cmake ..` from the repository root. (It’s also worth noting `${CUHFILES}` is referenced but never `set()` anywhere visible to this file, and would evaluate to empty; the “library” would consist only of the two header files listed, which — because they contain no non-template, non-inline definitions — would produce an essentially empty compiled `.a` even if this subdirectory were added.) This document flags this rather than assuming `src/CMakeLists.txt` plays an active role, since a straightforward reading of the directory-listing alone would suggest it does.

15.3 SlabAlloc/CMakeLists.txt — likewise not invoked from the SlabHash root

Similarly, `SlabAlloc` has its own `CMakeLists.txt` and its own `test/CMakeLists.txt` (`SlabAlloc/test/test_slab_alloc.cu`, a standalone smoke test of the allocator in isolation from any hash table) — these are meant for building and testing `SlabAlloc` **as its own independent project** (its README describes its own, separate `git clone` + `cmake` + `make` flow). When used as a submodule of `SlabHash`, only its **headers** (`SlabAlloc/src/slab_alloc.cuh`, `slab_alloc_global.cuh`) are consumed, via the root project’s `include_directories(SlabAlloc/src)` — `SlabAlloc`’s own build files are irrelevant to building `SlabHash`.

15.4 test/CMakeLists.txt and bench/CMakeLists.txt

```

# test/CMakeLists.txt
cuda_add_executable(cmap_test cmap_test.cu OPTIONS ${GENCODE} ${CUDA_VERBOSE_PTXAS})
target_link_libraries(cmap_test gtest)

cuda_add_executable(test_slab_hash test_slab_hash.cu OPTIONS ${GENCODE} ${CUDA_VERBOSE_PTXAS})
cuda_add_executable(concurrent_map concurrent_map.cu OPTIONS ${GENCODE} ${CUDA_VERBOSE_PTXAS})
cuda_add_executable(concurrent_set concurrent_set.cu OPTIONS ${GENCODE} ${CUDA_VERBOSE_PTXAS})
cuda_add_executable(iterator_test iterator_test.cu OPTIONS ${GENCODE} ${CUDA_VERBOSE_PTXAS})

# bench/CMakeLists.txt
add_library(random_gen batched_data_gen.cpp)
cuda_add_executable(benchmark main_benchmarks.cu OPTIONS ${GENCODE} ${CUDA_VERBOSE_PTXAS})
target_link_libraries(benchmark random_gen)

```

Five test executables (only `cmap_test` links against `gtest`; the other four are the standalone `main()`-based smoke tests from §14.2) and one benchmark executable (`benchmark`, linked against a small `random_gen` static library built from `bench/batched_data_gen.cpp`, §13.1’s data generator). `CMAKE_RUNTIME_OUTPUT_DIRECTORY` (set in the root file) means all of these end up in `build/bin/`.

15.5 How the repository is actually built and run

Following the README’s own instructions (§15.1’s “step 3” is the one step that requires a decision from the user, based on their actual GPU):

```

git submodule init
git submodule update
# edit CMakeLists.txt (or use ccmake ..) to enable the SLABHASH_GENCODE_SM**
# option matching your GPU's compute capability – SM35 is ON by default
mkdir build && cd build
cmake ..
make

```

This produces `build/bin/cmap_test`, `build/bin/test_slab_hash`, `build/bin/concurrent_map`, `build/bin/concurrent_set`, `build/bin/iterator_test`, and `build/bin/benchmark`. Running the `gtest` suite is then `./build/bin/cmap_test` (optionally with standard `gtest` flags like `--gtest_filter=...`); running a benchmark sweep is `./build/bin/benchmark -mode 0 [-num_key ... -expected_chain ... ...]` (§13.1’s flag list), or via the `bench/bencher.py` convenience wrapper.

16. Complete Call/Data-Flow Walkthroughs

§10 already gave a full, line-annotated trace of **search**. This section does the same for **insertion** and **deletion**, and then restates search in the same compact arrow-diagram format for side-by-side comparison.

16.1 Insertion: Application → GpuSlabHash API → Context → Kernel → Warp → Bucket → Slab → Key/Value

```
Application (e.g. test/cmap_test.cu's TEST(BulkBuild, IndividualSearch))
├─ cmap.hash_build(h_key.data(), h_value.data(), h_key.size())
└─
  └─ gpu_hash_table::hash_build [src/gpu_hash_table.cuh]
      └─ cudaMemcpy(d_key, h_key, ..., H2D); cudaMemcpy(d_value, h_value, ..., H2D)
          └─ slab_hash->buildBulk(d_key, d_value, num_keys)
              └─ GpuSlabHash::buildBulk [src/concurrent_map/cmap_implementation.cuh]
                  └─ num_blocks = ceil(num_keys / 128)
                      └─ build_table_kernel<<<num_blocks, 128>>>(d_key, d_value, num_keys, gpu_context_)
                          └─ build_table_kernel (__global_) [src/concurrent_map/device/build.cuh]
                              └─ tid, laneId computed; whole-warp early exit if out of range
                                  └─ AllocatorContextT local_allocator_ctx(slab_hash.getAllocatorContext());
                                      └─ local_allocator_ctx.initAllocator(tid, laneId); <-- allocator readied HERE
                                          └─ myKey, myValue = d_key[tid], d_value[tid]; myBucket = slab_hash.computeBucket(myKey)
                                              └─ slab_hash.insertPair(to_insert, laneId, myKey, myValue, myBucket, local_allocator_ctx)
                                                  └─ GpuSlabHashContext::insertPair (__device__, warp-cooperative) [src/concurrent_map/warp/insert.cuh]
                                                      └─ ballot work_queue; pick src_lane; shuffle its bucket to every lane
                                                          └─ every lane reads word `laneId` of the current slab (base, or `next`)
                                                              └─ isEmpty = ballot(src_unit_data == EMPTY_KEY) & REGULAR_NODE_KEY_MASK
                                                                  └─ isEmpty == 0 (slab is full of keys)
                                                                      └─ next_ptr = shuffle(src_unit_data, lane 31) <- read the "next" word
                                                                          └─ next_ptr == EMPTY_INDEX_POINTER (chain truly ends here):
                                                                              └─ new_node_ptr = allocateSlab(local_allocator_ctx, laneId)
                                                                                  └─ -> AllocatorContextT::warpAllocate(laneId) [SlabAlloc/
                                                                                      src/slab_alloc.cuh, §5.5 – bitmap ballot + atomicCAS]
                                                                                          └─ if (laneId==31): atomicCAS(next-word ptr, EMPTY_INDEX_PTR,
                                                                                              new_node_ptr) [§6.3, §12.1]
                                                                                                  └─ loss -> freeSlab(new_node_ptr) [§5.8]
                                                                                                      └─ (either way, loop repeats; winner's slab now in the chain)
                                                                                                          └─ next_ptr != EMPTY_INDEX_POINTER: next = next_ptr; loop repeats
                                                                                                              └─ reading THAT slab next iteration via getPointerFromSlab
                                                                                                                  └─ isEmpty != 0 (an empty key slot exists in THIS slab)
                                                                                                                      └─ dest_lane = first set bit of isEmpty
                                                                                                                          └─ if (laneId == src_lane):
                                                                                                                              └─ p = getPointerFromBucket(src_bucket, dest_lane) [or getPointerFromSlab]
                                                                                                                                  └─ old = atomicCAS((u64*)p, EMPTY_PAIR_64, packed(myKey,myValue)) [§6.4, §12.2]
                                                                                                                                      └─ if (old == EMPTY_PAIR_64): to_be_inserted = false <- SUCCESS, key/value stored
                                                                                                                                          └─ (loop continues until work_queue == 0, i.e. every lane in the warp has been serviced)
                                                                                                                                              └─ build_table_kernel returns -> GpuSlabHash::buildBulk returns -> gpu_hash_table::hash_build
                                                                                                                                                  └─ returns elapsed time; keys are now live in GPU global memory (d_table_and/or
                                                                                                                                                      SlabAllocLight::d_super_blocks_, depending on which slab each landed in)
```

Every arrow above corresponds to an actual function call or code branch cited in this document: **hash_build** (§13.5), **buildBulk** (§6.1/§6.5), **build_table_kernel** (§6.5), **insertPair** (§6.4–§6.5, quoted in full in §16.1's box above from **src/concurrent_map/warp/insert.cuh**), **warpAllocate** (§5.5), the link-in **atomicCAS** (§6.3, §12.1), and the key/value **atomicCAS** (§6.4, §12.2).

16.2 Deletion

```

Application
├─ cmap.hash_delete(h_deleted_keys.data(), num_deletion)
└─
  gpu_hash_table::hash_delete [src/gpu_hash_table.cuh]
  │ cudaMemcpy(d_key_, h_deleted_keys, ..., H2D)
  │ slab_hash->deleteIndividual(d_key_, num_deletion)
  └─
    GpuSlabHash::deleteIndividual [src/concurrent_map/cmap_implementation.cuh]
    │ delete_table_keys<<<num_blocks, 128>>>(d_key, num_deletion, gpu_context_)
    └─
      delete_table_keys (__global__) [src/concurrent_map/device/delete_kernel.cuh]
      │ tid, laneId, myKey, myBucket computed the same way as build/search
      │ (no allocator context needed – deletion never allocates)
      │ slab_hash.deleteKey(to_delete, laneId, myKey, myBucket)
      └─
        GpuSlabHashContext::deleteKey (__device__, warp-cooperative) [src/concurrent_map/warp/delete.cuh]
        │ ballot work_queue; pick src_lane; shuffle its key+bucket to every lane
        │ every lane reads word `laneId` of the current slab (base, or `next`)
        │   isFound = ballot(src_unit_data == src_key) & REGULAR_NODE_KEY_MASK
        │
        │ isFound == 0 (key not in this slab) ───────────────────────────
        │   next_ptr = shuffle(src_unit_data, lane 31)
        │   └─ next_ptr == EMPTY_INDEX_POINTER: to_be_deleted = false <- GIVE UP, not present
        │   └─ else: next = next_ptr; loop repeats on the next slab
        │
        │ isFound != 0 (key found at dest_lane) ───────────────────────────
        │   if (laneId == src_lane):
        │     p = getPointerFromBucket(src_bucket, dest_lane) [or getPointerFromSlab]
        │     old_pair = atomicExch((u64*)p, EMPTY_PAIR_64) [$6.7, $12.3]
        │     successful_deletion = (uint32_t)old_pair == src_key
        │     to_be_deleted = false
        │
        └─ (loop continues until every lane in the warp is done)
        delete_table_keys returns -> GpuSlabHash::deleteIndividual returns -> hash_delete returns
        the slot's memory now reads EMPTY_PAIR_64 (both key and value words = 0xFFFFFFFF);
        the SLAB ITSELF IS NOT RETURNED to the allocator (§5.8) – only its contents were cleared

```

16.3 Search, restated compactly for comparison (full trace in §10)

```

Application
├─ cmap.hash_search(h_query.data(), cmap_results.data(), num_queries)
└─
  gpu_hash_table::hash_search → GpuSlabHash::searchIndividual → search_table (__global__)
  │ [src/gpu_hash_table.cuh] [cmap_implementation.cuh] [device/search_kernel.cuh]
  └─
    GpuSlabHashContext::searchKey (__device__, warp-cooperative) [warp/search.cuh]
    │ ballot work_queue; pick src_lane; shuffle its key+bucket to every lane
    │ every lane reads word `laneId` of the current slab (base, or `next`)
    │   found_lane = ffs(ballot(src_unit_data == wanted_key) & REGULAR_NODE_KEY_MASK) - 1
    │   found_lane < 0 and lane-31's word == EMPTY_INDEX_POINTER -> myValue = SEARCH_NOT_FOUND
    │   found_lane < 0 and lane-31's word != EMPTY_INDEX_POINTER -> next = that word, loop
    │   found_lane >= 0 -> myValue = shuffle(src_unit_data, found_lane + 1) <- VALUE lane
    └─
      myResult written back to d_results[tid]

```

16.4 One structural observation across all three operations

Insert, search, and delete share **the exact same skeleton**: a `__ballot_sync`-driven `work_queue` loop, the same `A_INDEX_POINTER` / `EMPTY_INDEX_POINTER` sentinels, the same `getPointerFromBucket` / `getPointerFromSlab` dispatch on `next`, and the same `REGULAR_NODE_KEY_MASK`-masked ballot to find a relevant lane. The only differences are (a) what condition is being balloted for (`EMPTY_KEY` vs. matching the wanted key) and (b) what happens once a relevant lane is found (a 64-bit CAS to insert, a 64-bit exchange to delete, or a shuffle-and-copy to return a value). This is strong evidence that the warp-cooperative ballot-loop pattern (§7.2) is the actual core abstraction of this codebase — everything else is a thin variation on top of it.

17. Recommended Reading Order

This order is specific to SlabHash's actual dependency structure (§2.3), not a generic "read files alphabetically" suggestion.

1. README.md
Read this first because it states the paper this repository implements, the two-class ownership design (GpuSlabHash vs. GpuSlabHashContext), and the compilation steps – you need the ownership model in your head before any class definition will make sense.
2. src/slab_hash_global.cuh
Read this next because it defines every slab layout (concurrent_slab, key_only_slab, phase_concurrent_slab), every sentinel constant (EMPTY_KEY, EMPTY_INDEX_POINTER, A_INDEX_POINTER), and the SlabHashTypeT/ConcurrentMapT/ConcurrentSetT traits classes – nothing else in the codebase makes sense without these definitions in hand (§4).
3. SlabAlloc/src/slab_alloc_global.cuh, then SlabAlloc/src/slab_alloc.cuh
Read the allocator before the hash table that uses it, because GpuSlabHashContext's every insert/delete path calls straight into AllocatorContextT – you need warpAllocate/getPointerFromSlab/freeUntouched's mechanics (§5) in hand first, or the hash-table warp code will look like unmotivated bit-twiddling.
4. src/slab_hash_helper_methods.cuh
Short (three tiny functions), and it introduces the "ballot-then-mask-then-ffs" pattern (§7.2, §7.4) in its simplest form, used directly by ConcurrentSet and conceptually mirrored (inlined) everywhere in ConcurrentMap.
5. src/concurrent_map/cmap_class.cuh
This is where GpuSlabHashContext and GpuSlabHash are actually declared (§3, §8) – read it slowly; every member here is referenced constantly from this point on.
6. src/concurrent_map/warp/insert.cuh, then search.cuh, then delete.cuh, then count.cuh
In that order: insert.cuh introduces the full ballot-loop pattern with the most branches (empty slot vs. full-slab vs. chain-follow), so reading it first makes search.cuh and delete.cuh feel like simplifications rather than new material (§6, §7, §16).
7. src/concurrent_map/device/build.cuh, search_kernel.cuh, delete_kernel.cuh, count_kernel.cuh
Now that you know what insertPair/searchKey/deleteKey/countKey DO, read the kernels that CALL them – this is where tid/laneId, the whole-warp early exit, and the allocator-context setup pattern (§7.1, §5.10) live, tying the device-side operations back to an actual grid launch.
8. src/concurrent_map/cmap_implementation.cuh
The host-side glue – short, and now everything in it (kernel launches with computed grid sizes) will look obvious (§6.1, §6.5-6.7).
9. src/gpu_hash_table.cuh
The application-level wrapper every test and benchmark actually uses – read it now that you understand what it's wrapping (§3.1).
10. test/test_slab_hash.cu, then test/cmap_test.cu
Confirm your understanding against the simplest possible driver first, then the full gtest suite, which exercises every operation (§14).
11. src/concurrent_set/cset_class.cuh, cset_warp_operations.cuh, cset_implementation.cuh, cset_helper_kernels.cuh
Now that ConcurrentMap is second nature, ConcurrentSet reads as "the same pattern, minus values, plus the shared helper functions from step 4" (§9.2) – and its gaps (empty-bodied deleteIndividual, searchBulk, computeLoadFactor) will be obvious rather than surprising.
12. src/slab_iterator.cuh, then test/iterator_test.cu
A good way to double-check your mental model of "how does a traversal actually walk a bucket's chain" against independent code that does exactly that, outside of insert/search/delete.
13. bench/experiments.cuh, main_benchmarks.cu, batched_data_gen.h/.cpp
Read last – this is purely a consumer of everything above, useful mainly to see how num_buckets is chosen from an expected chain length (§13.2) and how mixed insert/delete/search workloads are driven via GpuSlabHash::batchedOperation.
14. CMakeLists.txt (root), src/CMakeLists.txt, test/CMakeLists.txt, bench/CMakeLists.txt
Read last, since the surprising fact that src/CMakeLists.txt and SlabAlloc/CMakeLists.txt are never invoked (§15) only makes sense once you already understand that everything in src/ is header-only templates.

18. A Note on the Code Snippets in This Document

Every code snippet throughout this document (§1–§17) has already followed the labeling convention requested: each is introduced with its file path and the function/class it belongs to (either in the paragraph immediately before it or in a comment on the snippet’s first line), copied verbatim from the cloned repository with no rewriting, and followed by prose explaining what it does and how it connects to the rest of the implementation. Rather than repeat that material a second time here, this section exists only to record that the requirement was applied throughout, not just in one place — if you are looking for a specific mechanism’s source, the section headings in §1–§17 map directly onto the mechanism (e.g. “the CAS that links a new slab” is in §6.3 and §12.1; “the bitmap allocation vote” is in §5.5; “the value-lane shuffle” is in §7.3 and §10.2).

19. SlabHash — Complete Mental Model

Replacing the prompt’s placeholder diagram with the architecture this document actually verified against the source, end to end:

HOST

```

Application code (test/*.cu, bench/*.cu, or your own program)
  constructs and calls the public API of:
  ↓
gpu_hash_table<KeyT, ValueT, SlabHashT> [src/gpu_hash_table.cuh]
  owns, via 'new':
  ↓
  → DynamicAllocatorT (= SlabAllocLight<...>) [SlabAlloc/slab_alloc.cuh]
    cudaMalloc's d super_blocks_ (ONE flat pool, 32 super-
    blocks, each holding 256 memory blocks of 1024 slabs,
    each memory block preceded by its own 32-word bitmap)
    exposes a small, copyable AllocatorContextT via
    getContextPtr()
  → GpuSlabHash<KeyT, ValueT, SlabHashT> [cmap_class.cuh /
    cudaMalloc's d_table_: ONE 128-byte base slab per bucket
    (the ONLY thing that makes num_buckets_ real)
    stores dynamic_allocator_ as a borrowed pointer
    at construction, calls gpu_context_.initParameters(...)
    which COPIES hash_x_/hash_y_/d_table_ AND a value-copy of
    the allocator's context into gpu_context_
    ↓
    gpu_context_ : GpuSlabHashContext<KeyT, ValueT, SlabHashT>
      OWNS NOTHING. Small, POD-like. Exists to be passed
      BY VALUE into every kernel launch.
      ↓
      <<<num_blocks, 128>>>(..., gpu_context_)
      (build_table_kernel / search_table / delete_table_keys /
      count_key / batched_operations)

```

DEVICE (GPU)

```

__global__ kernel
tid = threadIdx.x + blockIdx.x*blockDim.x; laneId = tid & 0x1F
whole-WARP early exit if (tid - laneId) >= num_items
(if inserting) local_allocator_ctx = copy of allocator context;
initAllocator(tid, laneId) -> picks a resident
memory block for THIS warp
myKey/myBucket = slab_hash.computeBucket(myKey)
computeBucket: (((hash_x_ ^ key) + hash_y_) % PRIME) % buckets
slab_hash.insertPair / searchKey / deleteKey / countKey(...)
↓
GpuSlabHashContext::<operation> - WARP-COOPERATIVE ballot loop:
while (work_queue = __ballot_sync(all-still-active-lanes)):
src_lane = first lane still needing work
every lane __shfl_sync's src_lane's bucket/key
every lane reads WORD `laneId` of ONE 128-byte slab:
first pass this src_lane: getPointerFromBucket(...)
-> (uint32_t*)d_table_ + bucket*32 + laneId
later passes (chain not exhausted): getPointerFromSlab
-> AllocatorContextT::addressDecoder(next) + laneId
-> offset into d_super_blocks_
__ballot_sync + REGULAR_NODE_KEY_MASK finds the relevant
key lane (empty / matching / -- this slab's 15 (map)
or 31 (set) key lanes, never the next-pointer lane)
ACT:
insert, empty slot found -> single-lane 64-bit atomicCAS
insert, slab full -> allocateSlab (warpAllocate,
a SECOND warp-wide ballot,
this time over BITMAP bits)
+ single-lane atomicCAS to
link the new slab in
search, match found -> shuffle value from lane+1
search, chain exhausted -> SEARCH_NOT_FOUND
delete, match found -> single-lane 64-bit atomicExch
count, every match -> __popc accumulate, keep going
if chain not exhausted and lane still unresolved: advance
`next` to the shuffled lane-31 word, loop again
(loop ends when EVERY lane in the warp has been resolved)

Underneath every allocation above: SlabAllocLightContext::warpAllocate
each lane inverts ITS OWN cached bitmap word, finds a free bit
warp ballots for which lanes have a free bit at all
first such lane attempts an atomicCAS on ITS bitmap word in
d_super_blocks_ (claiming one 128-byte memory unit = one slab)
result address (super_block<<27 | mem_block<<10 | lane<<5 | bit)
is shuffled to every lane in the warp
if the whole resident memory block is full: rehash to a new one

```

In one paragraph: an application owns an allocator and a `GpuSlabHash`; the allocator owns one big flat GPU memory pool organized as super-blocks of bitmap-tracked memory blocks, each holding many 128-byte “slabs”; the `GpuSlabHash` owns a separate, contiguous array of one 128-byte base slab per bucket. A key is mapped to a bucket by a simple $(a \wedge$

$\text{key} + b) \bmod p \bmod n$ hash. Every actual operation — insert, search, delete, count — is performed not by an individual thread but by an entire 32-thread warp acting together: the warp repeatedly agrees (via `__ballot_sync`) on whose query is “active,” fetches one 128-byte slab as 32 coalesced 4-byte words (one per lane), compares against the active query via another masked ballot, and either resolves it, follows a next-pointer to the next slab in the chain (reading it exactly the same way), or — for insertion, when a chain is full — asks the allocator for a fresh slab through the *exact same* warp-ballot pattern applied to a bitmap instead of a key, and links it in with a single lane’s atomic compare-and-swap. Every point where two warps could race — linking a new slab, writing a key/value pair, deleting one, claiming a free memory unit — is protected by exactly one atomic instruction whose “did I win” check is the *only* cross-warp coordination in the entire codebase; no locks, no fences, no critical sections. `ConcurrentMap` is the fully-realized version of this design; `ConcurrentSet` is the same design with values removed (and, verified by reading the actual function bodies rather than just the declarations, with bulk search, delete, and load-factor computation left unimplemented); `PhaseConcurrentMap` is a named-but-unbuilt placeholder. That is SlabHash.